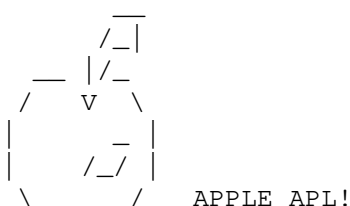


apple Extended Features Manual

*by Antonio Maschio
programmer in Montebelluna (TV) - Italy*

ABSTRACT



This is the manual for the extended features of **apple**, my APL engine. These notes follow a strict mathematical point of view, and if you should find errors and inconsistencies, please write to

ing dot antonio dot maschio at gmail dot com

I don't ask money for this program. I only ask for a voluntary contribution, to be made through PayPal (any amount), just to support my work and let me know you like it. Donations can be directed to

tbin at libero dot it

Thanks in advance to anyone of you who will contribute. And thanks anyway to anyone of you who will download and use **apple**! Remember: An **apple** a day keeps the doctor away!

11 April 2025

*This work is dedicated to the memory
of Kenneth E. Iverson (1920-2004)
Canadian computer scientist*

1. INTRODUCTION

APL is a mathematical language, no doubt. It can be seen as a programming language, and it is, indeed, but mathematics appears everywhere when you use it.

Linear algebra, on the other hand, is one of the disciplines that founded image processing and coordinates transformations. It's a subject I encountered in my first year of engineering in Padua, in the Academic Year 1985/86. Still, in this course, all matrix calculations were carried out by Professor Stagnaro on a blackboard, with white chalk, dust everywhere, small 3×3 matrices, but the magic was accomplished anyway. He was a big man with a big black beard, who used to say: *"If any one of you will ever invent anything useful for humanity, say something to make the blind see, I've my duty done"*. His book was hand-written and mimeographed (I still have it somewhere), with all examples invented by him. He was a superhero from the last non-electronic decade.

So, to make the two meet, to enhance the possibilities of this subject, and to study it again from a different prospective, I added to **apple** a bunch of operators that deal with the most advance topics in linear algebra: the QR Decomposition, the Hessenberg Decomposition, the eigen-decomposition, the Singular Value Decomposition and many others. These topics are performed by a small number of operators that return many results, that can be individually accessed by the axis mechanism (for instance the SVD operator `s [2] M` returns a different result than `s [3] M` or `sM`).

The table 2.9F lists the commands:

ISO operator	apple operator	Purpose	Notes
<absent>	a	matrix eigendecomposition	letter 'a' - indices 1-5
<absent>	b	matrix balancing decomposition	letter 'b' - indices 1-2
<absent>	d	matrix determinant	letter 'd'
<absent>	h	matrix Hessenberg decomposition	letter 'h' - indices 1-2
<absent>	j	matrix characteristic polynomial	letter 'j'
<absent>	k	matrix condition number	letter 'k'
<absent>	l	matrix triangular decompositions	letter 'l' - index 1 for the LL decomposition (Cholesky) - indices 2-3 for the LDL decomposition - indices 4-6 for the PLU decomposition - indices 7-9 for the LDU decomposition
<absent>	q	matrix QR Decomposition	letter 'q' - indices 1-2
<absent>	r	matrix echelon, RREF reduction and matrix rank	letter 'r' - indices 1-5
<absent>	s	matrix SV Decomposition	letter 's' - indices 1-7
<absent>	t	dyadic matrix tests	letter 't', left arguments 0-11
<absent>	w	matrix Polar Decomposition	letter 'w' - indices 1-3

Table 2.9F: Linear Algebra extended operators

I detached these notes from the manual, to let the user concentrate on what is more important for her/him. Some, but not all, of the features of these operators are also in the library files, so that in this case you can decide to use the APL versions, instead, and you're welcome. I only underline that these operators, coded in C, are much more faster

than the APL versions. Please note also that in the APL libraries, sometimes, these operators are used.

You are not forced to use them. You can build your own programs in APL, and they will have the same dignity of these commands, maybe more, because they'd be written in APL.

A final note: `apple` works only in $\Re$, so it does not treat complex numbers.

2. MATRIX DETERMINANT

The determinant of a matrix is a number that characterizes the matrix itself. In particular, the determinant is non-null iff the matrix is invertible, and is null for non-invertible matrices, and conventionally null for non-square matrices. In technical terms, the determinant is the last coefficient of the characteristic polynomial $p(x)$ (see the next paragraph about the characteristic polynomial operator j).

The monadic function dM returns the determinant of the matrix in M . Of course, in case of scalars and vectors a value error is raised.

Examples:

```

M
1  2  3
4  5  6
7  8  _9
dM
_54
d3
VALUE ERROR
d3
^
D
1  2  3
4  5  6
7  8  9
dD
0
N
1  2  3  4
5  6  7  8
dN
0

```

The determinant of the matrix inversion is automatically saved, after the execution of the operator $| \%$, if a scalar variable with the name DETERMINANT is found in the current variables space. But, in any moment, it can be calculated and returned by the monadic operator d , and returned as a scalar.

The following examples shows how DETERMINANT is influenced:

```

DETERMINANT<-0
| % | M
_1.72222    0.777778    _0.0555556
1.44444    _0.555556    0.111111
_0.0555556    0.111111    _0.0555556
DETERMINANT
_54

```

The assigning of zero to DETERMINANT is overwritten when $| \%$ M is executed, and afterwards DETERMINANT contains the value -54, which is the determinant of M .

3. MATRIX CHARACTERISTIC POLYNOMIAL

If one transformation is represented by a square matrix A of order $n \times n$, an eigenvector v and the correspondent eigenvalue λ , the three satisfy the relation:

$$A \cdot v = \lambda \cdot v$$

or in an equivalent fashion:

$$(\lambda I - A) \cdot v = 0$$

This has two consequences:

- 1 the matrix $(\lambda I - A)$ must be singular, and its determinant must be zero; in formula: $\det(\lambda I - A) = 0$
- 2 The eigenvalues of A may be interpreted as the roots of $\det(xI - A)$ which is a polynomial in x of degree n ; its coefficients define the characteristic polynomial, that *depicts* the matrix A in a special way; for instance, if the matrix is 4×4 , the polynomial is

$$a_1 x^4 + a_2 x^3 + a_3 x^2 + a_4 x + a_5 = 0$$

The coefficients, for what is in the polynomial, are $n + 1$, and so the values returned by `j` are a vector of dimension one more than the order of the matrix.

The monadic function `jM` returns the constants of the characteristic polynomial associated to a square variable in a vector that has as many items as the order of the square matrix plus one; in all other cases of non-square matrices, it returns the scalar zero, or a value error in case of non-matrices. For instance:

```

M
1  2  3
4  5  6
7  8  _9
    jM
1  3  _126  _54
```

It's worth noting that the last coefficient is also the determinant of the matrix:

```

dM
_54
```

and this shows why a non-invertible matrix, which has zero determinant, has an indeterminate characteristic polynomial, or, in other words, that at least one solution is zero:

$$a_1 x^4 + a_2 x^3 + a_3 x^2 + a_4 x = x(a_1 x^3 + a_2 x^2 + a_3 x + a_4) = 0$$

See also the matrix algorithm discussed about the matrix inversion operator `| % |` in the APL manual.

4. ECHELON AND ROW REDUCED FORMATS

The Echelon Format (EF) is the reduction of a matrix which eliminates all rows which are not linearly independent. The linear independence is a simple concept: two number sets are not linear dependent if there are no constants that multiplied by the first set yields the second set; for instance:

```
1 2 3
2 4 6
```

are linear dependent, because line 2 is derived from line 1 multiplied by 2, while

```
1 2 3
2 4 7
```

are not linear dependent (and thus are linear independent), because no constant can be multiplied by the first line to give the second (or vice versa), for all items. The echelon reduction has all ones on the main diagonal, as to reduce all lines to the simplest degree, and all null rows are pushed to the bottom and all null columns are pushed to the right.

The rank of a matrix (the word *rank* used in linear algebra is different from the meaning given to *rank* in APL) is the count of the linear independent rows (or columns) of a matrix, and if this count is lower than the order of the matrix, the matrix is not invertible (this is always the case of non-square matrices, because the rank can be at most the lower order, not the higher). Using the EF reduction, the rank becomes the minimum count of non-null rows or columns.

The EF reduction can be pushed to the maximum degree, to obtain the simplest version of the matrix that identifies its rank; this process is called Row Reduction Echelon Format, or RREF for short.

The function `rA` returns in its basic format the EF reduction of the object matrix *A*; the rank of the matrix is returned by `r[2]A`; the operators `r[3]A` and `r[4]A` return respectively the row and column ranks of *A* (the result of the operator `r[2]A` is always the lesser between the two).

The operator `r[5]A` returns the RREF version of the matrix *A*. This may or may not correspond to the EF reduction, but in any case the two versions share the same rank count for rows and columns.

Here are some examples; the first:

```

N
1  2  3  4
2  4  6  8
3  7  9  4
4  9 12  2

rN
1  2  3  4
0  1  0 _8
0  0  0  1
0  0  0  0

r\oN
1  2  3  4
0  0  1  1
0  0  0  1
0  0  0  0

r[3]N
```

3
 $r[4]N$
 4
 $r[5]N$
 $r[5] \setminus \circ N$
 $r[5] \setminus \circ N$

1	0	3	0
0	1	0	0
0	0	0	1
0	0	0	0

1	2	0	0
0	0	1	0
0	0	0	1
0	0	0	0

The second implies a very linear-dependent matrix:

W
 rW
 $r \setminus \circ W$
 $r[5]W$
 $r[5] \setminus \circ W$
 $r[3]W$
 $r[4]W$

1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4

1	2	3	4
0	0	0	0
0	0	0	0
0	0	0	0

1	1	1	1
0	0	0	0
0	0	0	0
0	0	0	0

1	2	3	4
0	0	0	0
0	0	0	0
0	0	0	0

1	1	1	1
0	0	0	0
0	0	0	0
0	0	0	0

1
 4

In the previous examples, the EF and RREF version coincide.


```

M
25    2    _3    4    5
 6    7    _8    9   10
11   12   _13   14   15
16   17   _18   19   20
21   22   _23   24   _25

rM
      1      0.08      _0.12      0.16      0.2
      0      1      _1.11656      1.23313      1.34969
      0      0      1      _2      _3
      0      0      0      0      1
      0      0      0      0      0

r\oM
      1      0.24      0.44      0.64      0.84
      0      1      1.70552      2.41104      3.11656
      0      0      1      2      3
      0      0      0      0      1
      0      0      0      0      0

r[5]M
1    0    0    0    0
0    1    0    _1    0
0    0    1    _2    0
0    0    0    0    1
0    0    0    0    0

r[5]\oM
1    0    0    0    0
0    1    0    _1    0
0    0    1    2    0
0    0    0    0    1
0    0    0    0    0

```

This reduction can be applied also on non-square matrices:

```

K
1    2    _1    _4
2    3    _1    _11
_2    0    _3    22

rK
1    2    _1    _4
0    1    _1    3
0    0    1    _2

r[3]K
3
r[4]K
4
r[2]K
3

```

```

r[5]K
1  0  0  _8
0  1  0  1
0  0  1  _2
r\oK
1  2  _2
0  1  _4
0  0  1
0  0  2
r[3]\oK
4
r[4]\oK
3
r[2]\oK
3
r[5]\oK
1  0  0
0  1  0
0  0  1
0  0  0

```

The RREF version often is the identity matrix, because this is the most reduced version for many matrices; here is an example that show this feature, unexpectedly:

```

A
1      2      5      5      5      5      7      8
0.5     1      2      4      5      5      8      5
0.2     0.5     1      2      2      5      5      5
0.2     0.25    0.5     1      3      4      8      5
0.2  0.333333  0.5  0.333333  1      5      5      5
0.2     0.2  0.333333  0.25    0.2     1      3      2
0.142857  0.125  0.2     0.125  0.2  0.333333  1      3
0.125     0.2     0.2  0.333333  0.2     5  0.333333  1
rA
1      2      5      5      5      5      7      8
0      1      0      10     10     40     36     34
0      0      1      _3     _5     _5     _9     _2
0      0      0      1  1.22222  _2.77778  _1  _3.11111
0      0      0      0      1      6.5     7.5     7.5
0      0      0      0      0      1  1.06513  1.20651
0      0      0      0      0      0      1  _1.87373
0      0      0      0      0      0      0      1
r[5]A
1  0  0  0  0  0  0  0
0  1  0  0  0  0  0  0
0  0  1  0  0  0  0  0
0  0  0  1  0  0  0  0
0  0  0  0  1  0  0  0

```

```
0 0 0 0 0 1 0 0
0 0 0 0 0 0 1 0
0 0 0 0 0 0 0 1
```

Last, but not least, the identity matrix is already in EF/RREF state (*ça va sans dire...*):

```
      I
1  0  0  0
0  1  0  0
0  0  1  0
0  0  0  1
```

```
    rI
1  0  0  0
0  1  0  0
0  0  1  0
0  0  0  1
```

```
    r[5]I
1  0  0  0
0  1  0  0
0  0  1  0
0  0  0  1
```

5. MATRIX CONDITION NUMBER

The condition number k , in numerical analysis, measures how much the output value of some function can change for a small change in the input argument. This is used to measure how sensitive a function is, and how much error in the output results from an error in the input. In our scopes, it estimates the precision that can be obtained in evaluating the eigenspace or the singular values of a target matrix, which requires usually a series of matrices multiplications.

A system is said to be *singular* if the condition number is infinite (which is also the case of non-invertible matrices), and *ill-conditioned* if it's anyway too large (where *too large* means roughly $\log_{10} C \geq p$, where p is the precision of matrix entries); it's *well-conditioned* if the number is close or equal to 1. As a general rule, the nearer is the condition number to 1, the more precision is obtainable. Wikipedia reports¹: "*As a rule of thumb, if the condition number $k(A) = 10^k$, then you may lose up to k digits of accuracy on top of what would be lost to the numerical method due to loss of precision from arithmetic methods. However, the condition number does not give the exact value of the maximum inaccuracy that may occur in the algorithm. It generally just bounds it with an estimate (whose computed value depends on the choice of the norm to measure the inaccuracy)*".

The condition number is a concept that can be applied to any mathematical problem. As for what concerns matrices, the definition is: in the expression $A \cdot x = b$, and posing A invertible, k is the maximum ratio of the relative error in x with the error in b .

The formula used for the calculation is the following, where σ identifies a singular value:

$$k(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}$$

For the calculation of $\sigma_{\max}$ and $\sigma_{\min}$ the SV Decomposition is applied on the target matrix A , and the maximum and minimum values are taken, which is precisely the task of the operator k .

Examples:

```

A
1  2  3
4  5  6
7  8  _9

sA
13.9876      0      0
      0  9.44355      0
      0      0  0.408805
    
```

The largest singular value is 13.9876, the smallest is 0.408805; their ratio is 34.2158, which is the condition number calculated for the matrix M .

```

kA
34.2158
    
```

I observe that Wolfram Alpha, for the same matrix, returns the condition number 61.3333, which does not correspond to the previous definition given, taken veritably from the Wolfram Alpha site². In any case the condition number for **apple** measures how precise the solutions in the eigenspace problem are, and for this purpose the numbers 34.2158 and 61.3333 are practically the same. The logarithm in base 10 of this number is:

```

10&kA
    
```

¹ https://en.wikipedia.org/wiki/Condition_number

² And precisely from here: <https://mathworld.wolfram.com/ConditionNumber.html>

1.53423

This value defines the loss of precision up to 1-2 digits of accuracy on top of what would be lost to the numerical method due to loss of precision from arithmetic method; in our case, assuming that the limit of computation is 10^{-15} , we expect exact results up to 10^{-13} at worst.

The next example features the calculation of the condition number for the identity matrix:

```
kI
1
```

This is the perfect condition number and this is valid for any order of the matrix I .

In case of singular matrices, the result is infinite, as shown in the next example:

```
M
1 2 3
4 5 6
7 8 9
kM
INF
```

5.1. Using the condition number

How is this number used? Let's see more examples: consider the following condition number:

```
M1
_1 2 3
4 5 6
7 8 9
kM1
123.34
10&kM1
2.09111
```

The condition number indicates a loss of precision of 2 decimals on top of the loss of the method used, and assuming 10^{-15} as the maximum precision adopted, it means we expect correct result up to 10^{-13} in the rebuild of the matrix M1; the actual error is about 10^{-12} , so the estimation is anyway good.

```
M1-(s[2]M1)+.x(sM1)+.x\os[3]M1

2.15383E_14 _1.26121E_13 9.76996E_14
2.25597E_13 _1.06493E_12 7.98472E_13
3.30402E_13 _1.77103E_12 1.29852E_12
```

The second operation follows:

```
M5
1 2 3
4 _5 6
7 8 9
kM5
```

```
14.6508
    10&kM5
1.16586
```

I think the difference between the two condition numbers is easy to detect, being the latter an order of magnitude smaller than the former. We expect here errors up to 10^{-14} . The error in rebuilding is this:

```
M5-(s[2]M5)+.x(sM5)+.x\os[3]M5

3.77476E_14  _7.54952E_15  _4.4853E_14
6.75016E_14           0  _4.79616E_14
1.79412E_13  _7.10543E_15  _1.22569E_13
```

The error is not less than 10^{-13} , which anyway attests the good estimation of the condition number.

A third case is the following:

```
M2

_1      2      3
4  14569      6
7      8      569
kM2
14044.6
    10&kM2
4.14751
    M2-(s[2]M2)+.x(sM2)+.x\os[3]M2

_3.30269E_12  2.66187E_12  _6.05738E_13
_3.74573E_08  1.45519E_11  4.24209E_10
3.3128E_10   _1.50813E_12  _4.20641E_12
```

In this case we expect a 4 digit loss, about 10^{-11} in our hypothesis. And actually, all the errors are close to the estimation, even if it must be noted that there is an error of 10^{-8} that probably ruins the correctness of rebuilding the matrix in some decomposition process. This is due to the delta in the values of M2.

The lesson is: the closer is the condition number to 1, the lesser is the error in rebuilding the matrix, the more precise are the results of the mathematical process. This is related to any matrix calculus, of course, from the SV, LU, QR decompositions to the Eigenspace determination, and pertains exquisitely to the nature of numbers... Something that goes beyond my power.

6. QR DECOMPOSITION

I've always been fascinated by the matrix decomposition algorithms, which take a matrix, disassemble it as they like and return two or three other matrices, different in content and properties, and then, when these results are recomposed, the original matrix comes back out! It's like a magic trick of the linear programming mathematics!

The QR Decomposition in `apple` is performed using the Gram-Schmidt algorithm, which takes a finite, linearly independent set of vectors $S = v_1, \dots, v_k$ for $k \leq n$ and generates an orthogonal set $S' = u_1, \dots, u_k$ that spans the same k -dimensional subspace of R_n as S . In practice, it tries to make two or more vectors perpendicular to each other.

This decomposition turns a matrix of any dimension into a product of two other matrices, that in order are one similar matrix Q and one upper triangular matrix R , so that:

$$A = Q \cdot R$$

The triangular matrix R has the property to be square (with dimensions equal to the number of columns of A) and always invertible.

Matrix Q has the same dimensions of the original matrix A , and may result orthogonal, in which case the following holds:

$$\begin{aligned} Q^{-1} &= Q^T \\ Q^T \cdot Q &= Q^{-1} \cdot Q = I \\ Q \cdot Q^T &= Q \cdot Q^{-1} = I \end{aligned}$$

so that the inverse is equal to its transpose. If R is complete, this is true, if R is not complete (i.e. it has some null rows or columns), this is not true.

The main scope of the QR Decomposition is to solve the *linear least square problem* (see ahead), and is the basic operation in the solution of eigenvectors and eigenvalues of a matrix in the QR algorithm (see the library programs).

The monadic operator `q` performs the QR Decomposition on the matrix argument, and returns in its basic usage the orthogonal matrix Q ; if used with the index 2, it returns the triangular matrix R . The matrices are returned in full form, so if Q or R have zero-columns or rows, they are maintained. This is necessary, because if `q` is needed for the QR algorithm, matrices should retain their original dimensions.

Here follow some examples of QR Decompositions:

```

M
1  2  3
4  5  6
7  8  _9
qM
0.123091  0.904534  _0.408248
0.492366  0.301511  0.816497
0.86164  _0.301511  _0.408248
q[2]M
8.12404  9.60114  _4.43129
      0  0.904534  7.23627
      0      0  7.34847
```

The proof that the decomposition works is here:

```
(qM) + .xq[2]M

1  2  3
4  5  6
7  8  _9
```

Lets verify that the orthogonal matrix is really orthogonal (we search for a confirmation, because qwe know it must be so, since q[2]M returned a fully triangular matrix):

```
| % | qM

0.123091  0.492366  0.86164
0.904534  0.301511  _0.301511
_0.408248  0.816497  _0.408248
\oqM

0.123091  0.492366  0.86164
0.904534  0.301511  _0.301511
_0.408248  0.816497  _0.408248
(qM) + .x\oqM

1  0  0
0  1  0
0  0  1
(\oqM) + .xqM

1  0  0
0  1  0
0  0  1
```

Here is another example:

```
M<-4 4pi16
M

1  2  3  4
5  6  7  8
9  10 11 12
13 14 15 16
qM

0.0601929  0.834492  0  0
0.300965  0.457625  0  0
0.541736  0.0807573  0  0
0.782508  _0.29611  0  0
q[2]M

16.6132  18.2986  19.9841  21.6695
0  1.07676  2.15353  3.23029
0  0  0  0
0  0  0  0
```


It can be observed that R is not perfectly triangular (while invertible), and thus Q is not orthogonal; nonetheless, the matrix M can be entirely rebuilt:

```
(qM) + .xq[2]M

1  2  3  4
5  6  7  8
9 10 11 12
13 14 15 16
```

The fact that the last two rows of R are made of nulls is a consequence of the fact that M is not invertible:

```
dM
0
```

The determinant is zero, that's how we know that M is not invertible. This leads to a non-fully triangular matrix R , so that the matrix Q itself is not orthogonal:

```
| % | qM

3.65692  0.813562      0      0
4.12348  0.205448      0      0
0.727672  7.32206  _3.86236      0
_2.66813  10.5763      0  _3.86236

\oqM

0.0601929  0.300965  0.541736  0.782508
0.834492  0.457625  0.0807573  _0.29611
0          0          0          0
0          0          0          0
```

The QR Decomposition, which is a splendid algorithm, never fails and works in any case, even with non-square matrices; see for instance the following example:

```
MM

3  5  9
2  1  _3
3  _5  2
2  4  _1
_1  2  8
4  1  1

qMM

0.457496  0.502413  0.457243
0.304997  0.0533287  _0.374599
0.457496  _0.704501  0.34219
0.304997  0.415403  _0.348942
_0.152499  0.275064  0.636846
0.609994  _0.0140339  _0.0785929

q[2]MM

6.55744  1.82998  3.20247
0  8.2856  4.7238
```

```

      0      0  11.2885
    (qMM) + .xq[2]MM

  3   5   9
  2   1  _3
  3  _5   2
  2   4  _1
 _1   2   8
  4   1   1

```

Here is the computation error estimation of the last example:

```

MM- (qMM) + .xq[2]MM

      0      0      0
      0      0      0
      0      0  2.22045E_16
      0      0  _3.33067E_16
      0      0  2.66454E_15
      0      0  1.11022E_16
+ / + / MM- (qMM) + .xq[2]MM
2.66454E_15
( + / ( + / (MM- (qMM) + .xq[2]MM) * 2) ) * .5
2.69672E_15

```

The first estimation is a simple sum over all the difference values, and this error estimation is 2.66×10^{-15} ; the second estimation is the square root of the sum of the squares of all values, and this totals 2.70×10^{-15} , which confirms substantially the first estimation; the error is thus about 3^{-15} and this is a very good result, to me.

6.1. Least Square Analysis

The usefulness of the QR Decomposition is shown by the Least Square Analysis³. If a linear system of equations is to be solved, and there are more equations than the occurrences of the unknowns variables (that is, there are more observation than needed), the classic equation:

$$M \cdot x = y$$

cannot solve the system giving the solutions from the equations

$$M^{-1} \cdot M \cdot y = M^{-1} \cdot y$$

$$y = M^{-1} \cdot y$$

simply because M is not invertible, and the pseudo-inverse is not useful here. But if we could decompose the original matrix M like this:

$$M = Q \cdot R$$

the problem would therefore be simplified as follows (provided Q is orthogonal and noticing that matrix R , if triangular, is always invertible):

$$M \cdot x = y$$

$$Q \cdot R \cdot x = y$$

$$Q^T \cdot Q \cdot R \cdot x = Q^T \cdot y$$

³ This paragraph extends the concepts exposed for the matrix inversion operator $| \%$ in the dyadic form (in the APL manual) and develops them further.

$$\begin{aligned} R \cdot x &= Q^T \cdot y \\ R^{-1} \cdot R \cdot x &= R^{-1} \cdot Q^T \cdot y \\ x &= R^{-1} \cdot Q^T \cdot y \end{aligned}$$

The solution is a simple matrix inner product (this solution is of course approximate, but this theory assures that it approximates the correct solution). Let's see an example in **apple**. The following matrix M has six rows, but only three columns. This means that we expect three solutions, but they must be interpolated along the six degrees polynomials describing the phenomenon. The mathematical input is the following $S(x)$ system of equations (this is the same example used in the dyadic inversion operator $| \% |$):

$$S(X) = \begin{cases} 3x_1 + 5x_2 + 9x_3 &= 0.6914727 \\ 2x_1 + x_2 - 3x_3 &= 2.4810138 \\ 3x_1 - 5x_2 + 2x_3 &= 0.4049580 \\ 2x_1 + 4x_2 - x_3 &= 0.3117301 \\ -x_1 + 2x_2 + 8x_3 &= 0.6084374 \\ 4x_1 + x_2 + x_3 &= 1.4778950 \end{cases}$$

This translates into a matrix problem where the principal matrix has more rows than columns (more equations than the involved variables which can count up to the columns number). Let's apply what we know so far:

```

M
3   5   9
2   1  _3
3  _5   2
2   4  _1
_1   2   8
4   1   1
Y
0.691473
2.48101
0.404958
0.31173
0.608437
1.47789

```

To prove the system is solvable we must have an orthogonal Q , that is a fully triangular R :

```

q[2]M
6.55744  1.82998  3.20247
      0   8.2856   4.7238
      0       0  11.2885

```

This is verified, so we can proceed with $x = R^{-1} \cdot Q^T \cdot y$:

```

( | \% | q[2]M ) + . x ( \ o qM ) + . x Y
0.322974
0.0725512
_0.0276467

```

The same is of course returned by the matrix division operator `| % |` that performs the Least Square analysis whenever the right argument is not invertible:

```
Y | % | M
0.322974
0.0725512
_0.0276467
```

7. Eigendecomposition

A scalar λ , real or complex, and a non-null vector x , form the *eigenspace* of matrix A if and only if:

$$A \cdot x = \lambda \cdot x$$

In this case, λ is called an **eigenvalue** of matrix A and x is called an **eigenvector** of matrix A .

Extending the reasoning to all the eigenvalues λ_i of A , for $i = 1 \dots n$, where n is the order (dimension) of the matrix A , and setting all these eigenvalues in the diagonal of a zero matrix J (so the matrix is diagonal), we have;

$$A \cdot P = P \cdot J$$

This leads to the two equivalences that constitute the Jordan decomposition:

$$A = P \cdot J \cdot P^{-1} \quad (1)$$

$$J = P^{-1} \cdot A \cdot P \quad (2)$$

As stated by Cleve B. Moler in his book "Numerical Computing with Matlab"⁴, there are two difficulties with the eigenvalues decompositions: the first is simply that this decomposition does not always exist, and in this case, no solution can be found. The second is that, even if such a decomposition exists, it might be computationally unreachable, or anyway time-consuming, requiring a stop at a certain point that would not return exact values in virtue of this (*exact* to the extents permitted by the numerical calculus, of course). Or worse, the combination of zeroes, in sparse matrices, may nullify columns or rows, that inevitably tend to nullify the whole results. In this second case, the solution is replacing *diagonal* with *triangular*, and using orthogonal transformations. This leads to the Schur Decomposition, also called *Schur Triangulation* (after the Russian mathematician Issai Schur), a matrix decomposition that converts an arbitrary square matrix to an upper triangular matrix S whose diagonal elements are the eigenvalues of the original matrix, and an orthogonal matrix T (actually the first stage exposed before with the QR algorithm), such as:

$$A = T \cdot S \cdot T^T \quad (3)$$

It's similar to the Jordan Decomposition, with the difference that in the Jordan decomposition the matrix J is diagonal, and the eigenvectors matrix P is normalized. The eigenvalues of A are on the diagonal of S (even the complex form is hidden in it). The result can be used in place of the Jordan Decomposition, with all the cautions of the case.

The decompositions performed by **apple** are accomplished in a multi-stage process:

- 1 The first stage checks if the matrix is diagonal: in this case the result is simple: (J and S are equal to A , and P and T are the identity matrix); the algorithm stops.
- 2 The second stage verifies that the application of the QR algorithm to find an upper triangular matrix S and the orthogonal matrix T works, by returning a correct diagonal J and a correct orthogonal P (nothing is done on S and T , that are retained as the result of the QR Algorithm. In particular, matrix S is *equivalent* to matrix A because they share the same eigenvalues, and since the eigenvalues of a triangular matrix lie on its diagonal, the eigenvalues are already calculated in most cases; in this case the algorithm stops, because the solution was found.
- 3 The third step is the calculation of the multiplicity of the eigenvalues. If it does not remain unitary for all (not all eigenvalues are distinct), then the algorithm stops and the results of the QR Algorithm are retained. In future versions, maybe this phase will be enhanced.
- 4 If all eigenvalues are distinct and the QR algorithm does not work (step 2), then all eigenvalues λ_i are collected from the diagonal of matrix S , and used to solve the equations of type:

$$(A - \lambda_i \cdot I) \cdot x_i = 0$$

⁴ you can download the chapter 10, related to the Jordan decomposition, here:
<https://epubs.siam.org/doi/10.1137/1.9780898717952.ch10>

with i ranging from 1 to n , if n is the dimension of the square matrix. Each vector x_i is a column in the eigenvector matrix P and each λ_i is the correspondent eigenvalue. The matrix J , populated with the calculated eigenvalues on the diagonal, is thus equivalent to matrix A ; the eigenvectors matrix P is also seen as the orthogonal change of base matrix so that the two expression (1) and (2) above are satisfied (we usually adopt the first, calling it 'rebuild of A ').

Using the previously described algorithm, **apple** can thus solve (though limitedly) the eigenvalue problem using the operator **a**: **aA** and **a [1] A** return the Jordan diagonal matrix J ; **a [2] A** returns the normalized eigenvectors matrix P ; **a [3] A** returns the upper triangular matrix S (called also *Schur matrix*); **a [4]** returns the orthogonal matrix T ; **a [5]** returns the number of iterations of the QR algorithm.

This operator works only on square matrices, and the result is not always correct, as for the what said earlier; to make the user aware, I have inserted two warnings; the first informs that the calculation involves complex results: "WARNING: THIS EIGENSPACE HAS COMPLEX RESULTS", but returns them intact so that the user can dispose of them at will. The second warning, if it detects that the Jordan matrix J or the eigenvector matrix P is a null-matrix, warns the user with the message "WARNING: CANNOT CALCULATE THIS EIGENSPACE" and does not return anything, because the results wouldn't be correct in any case (in this case, however, the Schur matrix S and the orthogonal matrix T are regularly returned).

As a leading note, the operator **a** works at its best with matrices that have eigenvalues with multiplicity one (so, you may ask, how do I know the multiplicity is one? Try finding J with **a**, this operator rarely fails in elaborating eigenvalues, due to the QR Algorithm, and look at it).

As a first example, let's consider the decomposition of a small symmetric matrix; symmetric matrices have the wonderful property to have only real eigenvalues and eigenvectors, so the algorithm is likely to always find correct solutions for this kind of mathematical items:

```

M
1  3  _5
3  2  4
_5 4  6
aM
9.66278      0      0
0      4.53744      0
0      0      _5.20022
a[2]M
_0.458582    28.6833    1.34672
0.342467    35.4885    _1.11665
1            1            1

```

The results are clear. Note that with **aM** (and accordingly **a [2] M**) the eigenvalues are ordered in descending order. The rebuild of the original matrix is:

```

(a[2]M) + .x (aM) + .x | % | a[2]M
1  3  _5
3  2  4
_5 4  6

```

As you can see, the result corresponds to the original matrix, so the rebuild is correct. Let's estimate the error (with **)FUZZ** set to zero):

$$M - (a[2]M) + .x(aM) + .x| \% | a[2]M$$

$$\begin{array}{rrr} 1.55431E_{-15} & _1.55431E_{-14} & _9.76996E_{-15} \\ 5.32907E_{-15} & _2.17604E_{-14} & 1.55431E_{-14} \\ _6.21725E_{-15} & 1.77636E_{-15} & 1.06581E_{-14}^5 \end{array}$$

The difference between the two results is not null, but anyway smaller than 1×10^{-13} , which is acceptable.

The Schur and orthogonal matrices are also easily calculated:

$$\begin{array}{l} a[3]M \\ \begin{array}{rrr} 9.66278 & 0 & 0 \\ 0 & _5.20022 & 0 \\ 0 & 0 & 4.53744 \end{array} \\ a[4]M \\ \begin{array}{rrr} 0.398003 & 0.668319 & 0.628445 \\ _0.297227 & _0.554147 & 0.777545 \\ _0.867899 & 0.496257 & 0.0219098 \end{array} \\ (a[4]M) + .x(a[3]M) + .x \backslash oa[4]M \\ \begin{array}{rrr} 1 & 3 & _5 \\ 3 & 2 & 4 \\ _5 & 4 & 6 \end{array} \end{array}$$

As you can see, the rebuild is again perfect; the orthogonality of Q is soon proved:

$$\begin{array}{l} | \% | a[4]M \\ \begin{array}{rrr} 0.398003 & _0.297227 & _0.867899 \\ 0.668319 & _0.554147 & 0.496257 \\ 0.628445 & 0.777545 & 0.0219098 \end{array} \\ \backslash oa[4]M \\ \begin{array}{rrr} 0.398003 & _0.297227 & _0.867899 \\ 0.668319 & _0.554147 & 0.496257 \\ 0.628445 & 0.777545 & 0.0219098 \end{array} \end{array}$$

Another symmetric example:

$$\begin{array}{l} M \\ \begin{array}{rrrr} 1 & 2 & 3 & 4 \\ 2 & _1 & 0 & 1 \\ 3 & 0 & 1 & 2 \\ 4 & 1 & 2 & 7 \end{array} \\ aN \\ \begin{array}{rrrrr} 10.3852 & 0 & 0 & 0 & 0 \\ 0 & 1.23607 & 0 & 0 & 0 \\ 0 & 0 & _0.385165 & 0 & 0 \\ 0 & 0 & 0 & 0 & _3.23607 \end{array} \end{array}$$

⁵ You may see different results, of course...

```

a[2]N
0.596291 _0.763932 _2.09629 _5.23607
0.192582 _0.236068 _5.19258 4.23607
0.403709 _1.23607 3.09629 3.23607
      1      1      1      1
(a[2]M) + .x (aM) + .x | % | a[2]M

      1      2      3      4
      2      _1      1.11022E_15      1
      3 _1.22125E_14      1      2
      4      1      2      7

```

The precision is again up to 10^{-13} , which is satisfactory.

The Jordan Decomposition, when succeeding, is a numerically stable algorithm, because it's made of successive matrix multiplications, and although the QR algorithm is formally an infinite sequence of operations, convergence to machine precision is practically achieved in $O(n^3)$ operations (it turns out to be 20 to 1000 iterations at most for symmetric matrices, more for non-symmetric matrices).

From a numerical point of view, extending and categorizing the problems highlighted by Mr. Moler, the failure of the QR algorithm depends practically on many factors:

- The convergence may be very slow; if after 10000 iterations (the superimposed limit in the code) the result is not reached, going further is meaningless, so the cycle stops, but the result may be not achieved.
- The algorithms reaches the alternation of two results whose difference is too big to be ignored, the first leading to the second leading to the first and so on, without converging; the introduction of the limit of 10000 iterations prevents from going on infinitely, but again the result is not achieved.
- In some matrices with a diagonal of nulls, the QR algorithm produces a wrong Schur matrix, with nulls on the diagonal; this causes the eigenvalues to be interpreted as a series of null values, even if the relative polynomial has non-null roots; this causes the generation of null or wrong eigenvectors, and in any case the failure of the Schur or the Jordan decompositions is behind the corner.
- Another limitation of the QR algorithm is that the first stage generates complete fill-in in general sparse matrices. It can therefore not be applied to large sparse matrices, simply because of excessive memory requirements. On the other hand, the QR algorithm computes all eigenvalues (and eventually eigenvectors) which is rarely desired in sparse matrix computations anyway⁶.

So the QR algorithm solves some, but not all the eigenspace problems. And I think it's not worth the trouble the refining of the algorithm because **apple** can hardly be used for solving large matrices problems.

Let's see now some more complicated examples; the next is the classic non-symmetric matrix *A*:

```

A
1  2  3
4  5  6
7  8  _9
aA
10.0593      0      0
      0 _0.424882      0

```

⁶ Take a look at <https://people.inf.ethz.ch/~arbenz/ewp/Lnotes/chapter4.pdf>.


```

      0      0  _12.6344
a[2]A

0.718345    15.8213  _0.175977
1.75386    _12.7717  _0.300327
      1      1      1
a[3]A

_12.6344    3.51036  _2.10616
      0    10.0593  _2.69099
      0      0  _0.424882
a[4]A

_0.166196    0.364914  _0.916088
_0.283635    0.872058    0.398832
0.944421    0.326118  _0.0414303
(a[2]A)+.x(aA)+.x|a[2]A

1  2  3
4  5  6
7  8  _9
(a[4]A)+.x(a[3]A)+.x\o a[4]A

1  2  3
4  5  6
7  8  _9

```

The Jordan and Schur decompositions are both correct, because both reconstruct the original matrix⁷.

Sometimes the precision is not good, but the rebuild is, and this baffles me. See this example:

```

M

5  4  2  1
0  1  _1  _1
_1  _1  3  0
1  1  _1  2
aM

4.0004      0      0      0
      0  3.9996      0      0
      0      0      2      0
      0      0      0      1

```

The eigenvalues are quite correct. The literature (Wolfram Alpha) shows that they should be {4, 4, 2, 1}, so that the precision is not optimal, but the four eigenvalues are recognizable. The problem is that they are not in Jordan form, that is with ones in the super-diagonal to identify the multiplicity.

What are the eigenvectors, anyway? Here they are:

```
a[2]M
```

⁷ This is not achieved by the QR algorithm alone, which returns a wrong result, but by the union of the QR algorithm and the reverse equations solutions. See also the behaviour of the program JORD which uses a pure QR algorithm.

1.0004	0.9996	1	_0.017371
3.55229E_12	_3.55367E_12	_1	0.017371
_1	_1	0	0
1	1	1	0

These eigenvectors, though not optimal, rebuild the original matrix:

(a[2]M) + .x (aM) + .x % a[2]M			
5	4	2	1
1.77669E_08	1	_1	_1
_1	_1	3	0
1	1	_1	2

This shows that the eigenvalues and eigenvectors are basically good, though not officially lined to the Jordan form. The next example is a 5×5 non-symmetric matrix that has not full rank (4):

M

25	2	_3	4	5
6	7	_8	9	10
11	12	_13	14	15
16	17	_18	19	20
21	22	_23	24	_25

aM

33.6461	0	0	0	0
0	13.6766	0	0	0
0	0	0	0	0
0	0	0	_0.777386	0
0	0	0	0	_33.5453

There is one null eigenvalue, and all eigenvalues have multiplicity one.

a[2]M

1.19292	_1.35418	0	2.10991	_0.0666928
0.878016	1.93059	_0.000148012	34.6067	_0.208437
1.41403	2.83902	_0.000296023	1.96512	_0.302465
1.95004	3.74745	_0.000148012	_30.6765	_0.396493
1	1	0	1	1

(a[2]M) + .x (aM) + .x | % | a[2]M

25	2	_3	4	5
6	7	_8	9	10
11	12	_13	14	15
16	17	_18	19	20
21	22	_23	24	_25

The Jordan decomposition is correct, even if the rank is 4 and this is shown by the zero column in $J(aM)$. The Schur decomposition, instead, is not good:

a[3]M

33.6461	9.20275	22.6476	_3.70486	0
---------	---------	---------	----------	---

```

_1.07067E_12    _33.5453    12.9908    _1.75848    0
0              0          13.6766    0.902336    0
0              0              0    _0.777386    0
0              0              0        0        0

```

a[4]M

```

0.397727  _0.00464325    0.915715  _0.0570768    0
0.292735  _0.144666    _0.179818  _0.833293    0
0.471445  _0.203538    _0.215309  _0.152604    0
0.650154  _0.262409    _0.250799    0.528085    0
0.333405    0.932075    _0.140973  _0.0142703    0

```

(a[4]M)+.x(a[3]M)+.x\oa[4]M

```

25  1.23013E_12    1          2          5
6    1.66667    2.66667    3.66667    10
11   3.33333    4.33333    5.33333    15
16           5          6          7          20
21   6.66667    7.66667    8.66667    _25

```

The reconstruction is good only in the first and fifth columns, and completely wrong in the other columns, even if S seems correct, having the eigenvalues of M in the diagonal; the problem lies, in all likelihood, in the incorrect determination of the orthogonal matrix Q , which is not orthogonal, as proved by the following tests:

| % | a[4]M

```

0.606188    0.451676    _0.903353    2.25838    0
0.767914    _1.10542    0          0.903353    0
0.792264    _1.80671    0.249614    0.903353    0
0.816615    _2.71006    _3.61341    0.249614    0
_22.4698  _2.65743E+16  5.31486E+16  _2.65743E+16  _442.393

```

\oa[4]M

```

0.397727    0.292735    0.471445    0.650154    0.333405
_0.00464325  _0.144666    _0.203538  _0.262409    0.932075
0.915715    _0.179818    _0.215309  _0.250799    _0.140973
_0.0570768  _0.833293    _0.152604    0.528085    _0.0142703
0           0           0           0           0

```

The output is completely out of tune, and this may depend on the defective matrix M ; its echelon form is actually:

rM

```

1      0.08    _0.12    0.16    0.2
0      1    _1.11656  1.23313  1.34969
0      0      1      _2      _3
0      0      0      0      1
0      0      0      0      0

```

In case of impossible matrices, the result is even not returned. A very-well known difficult matrix is the following:

C

```

0  2  0  _1
1  0  0  0

```

```

0 1 0 0
0 0 1 0
  |%|C
0 1 0 0
0 0 1 0
0 0 0 1
_1 0 2 0

```

This matrix is rather sparse, but has only nulls in the diagonal. It's a cute matrix whose inverse is simply a double mirrored image (flipped up-down and left-right) of itself. But the eigenspace is forbidden. If we try to calculate the eigenvalues we get:

```
aC WARNING: CANNOT CALCULATE THIS EIGENSPACE
```

There's no results, because the QR algorithm fails completely (the matrix J is null).

Another interesting case is the matrix of ones:

```

F<-4 4p1
F

1 1 1 1
1 1 1 1
1 1 1 1
1 1 1 1
  aF

4 0 0 0
0 0 0 0
0 0 0 0
0 0 0 0
  a[2]F

0.5 0 0 0
0.5 0 0 0
0.5 0 0 0
0.5 0 0 0
  a[3]F

4 0 0 0
0 0 0 0
0 0 0 0
0 0 0 0
  a[4]F

0.5 0 0 0
0.5 0 0 0
0.5 0 0 0
0.5 0 0 0
  (a[4]F)+.x(a[3]F)+.x\oa[4]F

1 1 1 1
1 1 1 1

```

```
1 1 1 1
1 1 1 1
```

As you can see the Jordan and Schur decomposition work, even if the eigenvalues are correct (one is 4 with multiplicity one, and the second is zero with multiplicity three; it's just the zero case with multiplicity greater than one that makes fails the making of P)⁸.

The last peculiar case that I propose is:

```

B
  7   3   4  _11  _9  _2
 _6   4  _5   7   1  12
 _1  _9   2   2   9   1
 _8   0  _1   5   0   8
 _4   3  _5   7   2  10
  6   1   4  _11  _7  _1

aB
WARNING: THIS EIGENSPACE HAS COMPLEX RESULTS

  5  0  0  0  0  0
  0  5  0  0  0  0
  0  0  4  0  0  0
  0  0  0  3  0  0
  0  0  0  0  1  0
  0  0  0  0  0  1

a[2]B
WARNING: THIS EIGENSPACE HAS COMPLEX RESULTS

  0   0   1   0   0   0
  0   0  8.8   0   0   0
  0   0 _7.4   0   0   0
  0   0 _7.4   0   0   0
  0   0  8.8   0   0   0
  0   0   1   0   0   0

```

The fact that there are complex eigenvalues or eigenvectors makes fail the QR algorithm, and the eigenvalues and eigenvectors are not correct. In this case, we should rely on what Mr. Moler suggests, using *triangular* in place of *diagonal*:

```

a[3]B
WARNING: THIS EIGENSPACE HAS COMPLEX RESULTS

  5   _6  _6.96257  14.7798  12.5923  _1.24393
  6   5   2.12666  _12.2701  _6.98389  14.6667
  0   0   4   _3  _12.3961  2.88715
  0   0   0   3  _3.9437  _10.698
  0   0   0   0   1   2
  0   0   0   0  _2   1

a[4]B
WARNING: THIS EIGENSPACE HAS COMPLEX RESULTS

```

⁸ The program JORD in the library, which uses the QR algorithm only, is even worse, as you can see in the `apple` manual, and fails with both the Jordan and Schur decompositions.

```

0.70699  _0.0128278      0      0      0.373549  _0.600384
0.0128278      0.70699      0      0      0.600384      0.373549
0      0      0.707107      0.707107      0      0
0      0      0.707107  _0.707107      0      0
0.0128278      0.70699      0      0  _0.600384  _0.373549
0.70699  _0.0128278      0      0  _0.373549      0.600384
(a[4]B)+.x(a[3]B)+.x\oa[4]B
WARNING: THIS EIGENSPACE HAS COMPLEX RESULTS
7      3      4      _11      _9      _2
_6      4      _5      7      1      12
_1      _9      2      2      9      1
_8  3.56923E_11      _1      5  3.58922E_11      8
_4      3      _5      7      2      10
6      1      4      _11      _7      _1

```

The rebuild of matrix B is quite good (there are two negligible not-null values that are of order 10^{-11} , but we are quite sure the matrix S and the matrix T are correct. So, what now? Read on.

7.1. Determination of complex eigenvalues

The upper triangular matrix S returned by `a[3]` can be used for the calculation of the real and the imaginary parts of the complex eigenvalues because, in this case, the matrix is not properly triangular but has some non-null value under the diagonal.

Let's see what happens in the analysis of the last matrix B we encountered; let's calculate the Schur matrix again:

```

a[3]B
WARNING: THIS EIGENSPACE HAS COMPLEX RESULTS
5      _6  _6.96257      14.7798      12.5923  _1.24393
6      5      2.12666  _12.2701  _6.98389      14.6667
0      0      4      _3  _12.3961      2.88715
0      0      0      3  _3.9437  _10.698
0      0      0      0      1      2
0      0      0      0      _2      1

```

The Schur matrix is not fully triangular: the item under the elements (1,1) and (5,5) are not null; let's take into account the 2×2 sub-matrix starting from (1,1) first:

$$s_{1,2} = \begin{bmatrix} x_{1,1} & x_{1,2} \\ x_{2,1} & x_{2,2} \end{bmatrix}$$

Now, setting $b = x_{1,1} + x_{2,2}$, and $c = x_{1,1}x_{2,2} - x_{1,2}x_{2,1}$, the following holds:

- * The real part of the conjugated pair is $\frac{b}{2}$.
- * The complex parts are $\pm \frac{\sqrt{|b^2 - 4c|}}{2}$. One of the conjugated pair has the positive part, the other the negative.

Does it seem familiar? It should, it's the same formula for the solution of a second degree equation

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

where $a = 1$ and the argument of the square root is taken as absolute. In case of non-symmetric matrices, the eigenvalues are calculated in this way, and this conforms (for the most cases) to the results given by Wolfram Alpha. Just to prove the example, we have:

$$\begin{aligned}x_{1,1} &= 5 \\x_{2,1} &= 6 \\x_{1,2} &= -6 \\x_{2,2} &= 5\end{aligned}$$

$$\begin{aligned}b &= 5 + 5 = 10 \\c &= (5 \times 5) - (6 \times -6) = 61\end{aligned}$$

From these data we retrieve:

$$\begin{aligned}\text{real part} &= \frac{b}{2} = \frac{10}{2} = 5 \\ \text{complex parts} &= \pm \frac{\sqrt{|b^2 - 4c|}}{2} = \pm \frac{\sqrt{|100 - 244|}}{2} = \frac{\pm 12}{2} = \pm 6\end{aligned}$$

The item under the element (5,5) instead is:

$$s_{1,2} = \begin{bmatrix} x_{5,5} & x_{5,6} \\ x_{6,5} & x_{6,6} \end{bmatrix}$$

With the following values:

$$\begin{aligned}x_{5,5} &= 1 \\x_{5,6} &= 2 \\x_{6,5} &= -2 \\x_{6,6} &= 1\end{aligned}$$

we get:

$$\begin{aligned}b &= 1 + 1 = 2 \\c &= (1 \times 1) - (2 \times -2) = 5\end{aligned}$$

and from these data we retrieve:

$$\begin{aligned}\text{real part} &= \frac{b}{2} = \frac{2}{2} = 1 \\ \text{complex parts} &= \pm \frac{\sqrt{|b^2 - 4c|}}{2} = \pm \frac{\sqrt{|4 - 20|}}{2} = \frac{\pm 4}{2} = \pm 2\end{aligned}$$

All the eigenvalues of matrix B, including the real ones, are:

$$\begin{aligned}\lambda_1 &= 5 + 6i \\ \lambda_2 &= 5 - 6i \\ \lambda_3 &= 4 \\ \lambda_4 &= 3 \\ \lambda_5 &= 1 + 2i \\ \lambda_6 &= 1 - 2i\end{aligned}$$

Let's see another practical example; in the following, M1 is an invertible non-symmetric 4×4 matrix:

```

M1<-4 4p1 2 3 4 2 3 4 1 5 3 2 1 4 5 2 1
M1

1 2 3 4
2 3 4 1
5 3 2 1
4 5 2 1
a[5]M1
10001
a[3]M1
WARNING: THIS EIGENSPACE HAS COMPLEX RESULTS

10.6104      2.14561      1.00157      _1.95078
0      _4.01994      _0.571957      0.538421
0      0      _0.0759171      1.77168
0      0      _1.68818      0.485467

```

Again the matrix S is not fully triangular: the item under the element (3,3) is not null; let's take into account the 2×2 sub-matrix starting from (3,3):

$$s_{1,2} = \begin{bmatrix} x_{3,3} & x_{3,4} \\ x_{4,3} & x_{4,4} \end{bmatrix}$$

From:

$$\begin{aligned}
 x_{3,3} &= -0.0759171 \\
 x_{4,3} &= -1.68818 \\
 x_{3,4} &= 1.77168 \\
 x_{4,4} &= 0.485467
 \end{aligned}$$

we get:

$$\begin{aligned}
 b &= -0.0759171 + 0.485467 = 0.409549 \\
 c &= (-0.0759171 \times 0.485467) - (-1.68818 \times 1.77168) = 2.95406
 \end{aligned}$$

From these data we finally retrieve:

$$\begin{aligned}
 \text{real part} &= \frac{b}{2} = 0.204775 \\
 \text{complex parts} &= \pm \frac{\sqrt{|b^2 - 4c|}}{2} = \pm \frac{\sqrt{|0.167731 - 11.8162|}}{2} = \pm \frac{3.41299}{2} = \pm 1.70650
 \end{aligned}$$

All the eigenvalues are:

$$\begin{aligned}
 \lambda_1 &= 10.6104 \\
 \lambda_2 &= -4.01994 \\
 \lambda_3 &= 0.204775 + 1.70650i \\
 \lambda_4 &= 0.204775 - 1.70650i
 \end{aligned}$$

All real/complex eigenvalues in this paragraph were verified positively against the Wolfram Alpha engine.

7.2. Utility and versatility of eigenvectors and eigenvalues.

According to Wikipedia⁹, "Eigenvalues and eigenvectors feature prominently in the analysis of linear transformations. The prefix eigen- is adopted from the German word eigen (cognate with the English word own) for 'proper',

'characteristic', 'own'. Originally used to study principal axes of the rotational motion of rigid bodies, eigenvalues and eigenvectors have a wide range of applications, for example in stability analysis, vibration analysis, atomic orbitals, facial recognition, and matrix diagonalization".

Such broad fields of applications make a mathematical environment like APL\1130 (and **apple**) hungry for data like these. This is why I wrote the library.

An interesting field of application involves the diagonalization of a matrix, that is the decomposition of a matrix into a diagonal J and a conversion matrix P , which is exactly the process described for the QR algorithm; in this regard, the diagonal matrix J is what is called the *diagonalization* Δ of the original matrix A , and the eigenvectors matrix P is what is called the *change of basis matrix* P associated to the diagonal restitution, so that

$$A = P[\text{md}]\Delta \cdot P^{-1}$$

which is the same rebuild shown for the QR algorithm.

Some notable properties of the eigenvalues are:

- The trace of matrix $A = a_{ij}$, $i, j = 1 \dots n$ is the sum of all the eigenvalues λ_i , $i = 1 \dots n$; this property can be used to test if the returned eigenvalues are correct:

$$\text{tr}(A) = \sum_{i=1}^n a_{ii} \Rightarrow \sum_{i=1}^n \lambda_i = \lambda_1 + \lambda_2 + \dots + \lambda_n$$

See for instance the program EIGTEST in the library.

- The determinant of matrix $A = a_{ij}$, $i, j = 1 \dots n$ is the negative product of all the eigenvalues λ_i , $i = 1 \dots n$; again, this property can be used to test if the returned eigenvalues are correct:

$$\det(A) = - \prod_{i=1}^n \lambda_i \Rightarrow - \lambda_1 \lambda_2 \dots \lambda_n$$

See for instance the program EIGTEST in the library.

- The eigenvalues of a diagonal matrix are the element on the diagonal. From this, we derive that if the process of diagonalization produces a diagonal matrix that has the same eigenvalues spectrum of the original matrix, the eigenvalues of the original matrix can be found on the diagonal of its diagonalized version.
- The matrix A is invertible if all its eigenvalues are not null; this is practically demonstrated by the determinant which is the multiplication of all the eigenvalues: if any is zero, the final result is zero; if the final result is not zero, no eigenvalue is zero. The Gaussian theory, on the other hand, attests that non-invertible matrices return a null determinant, and this is the mirrored side of the same assertion.
- If A is invertible, then the eigenvalues of A^{-1} are $\frac{1}{\lambda_1}, \frac{1}{\lambda_2} \dots \frac{1}{\lambda_n}$
- If A is Hermitian (that is equal to its conjugate transpose), then every eigenvalue is real. In the world of reals, this is call symmetry, and so, if the matrix is symmetric, all its eigenvalues are real.
- If A is unitary, each eigenvalue respects the condition $|\lambda_i| = 1$. In particular, the identity matrix has only one eigenvalue 1 with multiplicity equal to the order of the matrix.
- The eigenvalues are the solution of the characteristic polynomial of the matrix (see also the **apple** operator `j`); again, this can be used to test if the eigenvalues are correct by solving the equation, using each eigenvalue as a solution of the equation itself.

There are many more properties, related to the eigenvalues, but the ones reported here are the ones that are useful in

⁹ https://en.wikipedia.org/wiki/Eigenvalues_and_eigenvectors

the practical calculations.

A fascinating field of application is to see the differential operator in parallel with the eigenspace theory; for example, the following expression of a linear transformation with differential operator:

$$\frac{d}{dx} e^{\lambda x} = \lambda e^{\lambda x}$$

is representable with a $n \times n$ matrix, that obeys to the law:

$$A \cdot v = \lambda \cdot v$$

where each vector v is one column of the normalized matrix P returned by the eigenvectors programs, and each λ is the correspondent eigenvalue, and so each eigenvector is a solution for $e^{\lambda x}$ and each eigenvalue is a solution for λ .

7.3. The Rows Geometric Mean Method

The Rows Geometric Mean Method (RGMM) tries to find the eigenvector W_i corresponding to the main eigenvalue $\lambda_{\max}$ using geometric means. The method is used in hydrogeology, where the eigenvector shows the order of influence of the factor of subterranean water recharge, by assigning and calculating the relative weights W_i of each parameter, with reference to a pairwise comparison matrix A . This pairwise comparison matrix is a special matrix for some physical parameters like rainfall, soil type, slope, depth to water table and so forth.

The eigenvalue $\lambda_{\max}$ associated to W_i is used to determine the Consistency Index CI of the matrix, which in some sense attests if the matrix has all the needed parameters, with the formula:

$$CI = \frac{\lambda_{\max} - n}{n - 1}$$

and from this the Consistency Ratio CR is derived:

$$CR = \frac{CI}{RCI}$$

where RCI is some consistency index, which depends on the matrix size (I don't expect you understand this, it's technical stuff that is arabic to me too), fixed to 1.41 for an 8×8 matrix.

The interest in this method lies in the way used to calculate W_i and $\lambda_{\max}$. Let's define the following geometric means r_i and r_j :

$$r_i = \left(\prod_{j=1}^n a_{ij} \right)^{\frac{1}{n}}$$

and

$$r_j = \left(\prod_{i=1}^n a_{ij} \right)^{\frac{1}{n}}$$

The eigenvector is calculated this way:

$$W_i = \frac{r_i}{\sum_{j=1}^n r_j}$$

The relative eigenvalue can be derived from

$$A \cdot W_i = \lambda_{\max} W_i$$

and so it is:

$$\lambda_{\max} = \frac{1}{n} \sum_i^n \frac{(A \cdot W)_i}{W_i}$$

The validity of the pairwise comparison matrix is in relation with the value of $\lambda_{\max}$: if $\lambda_{\max} \geq n$ (where n is the number of parameters involved in the study) then the matrix is good; otherwise, a new matrix is required (i.e. more parameters are needed)¹⁰.

Let's work in APL. The matrix A is built for the following parameters: Rainfall, Soil Type, Land Use/Land Cover, Slope, Unsaturated Zone, Drainage Density, Depth to water table, Curve number ($n = 8$); the matrix is hypo-symmetric¹¹, that is it respects the condition:

$$a_{ij} = \frac{1}{a_{ji}}$$

The same example in the cited article is reported here:

A							
1	2	5	5	5	5	7	8
0.5	1	2	4	3	5	8	5
0.2	0.5	1	2	2	3	5	5
0.2	0.25	0.5	1	3	4	8	3
0.2	0.333333	0.5	0.333333	1	5	5	5
0.2	0.2	0.333333	0.25	0.2	1	3	2
0.142857	0.125	0.2	0.125	0.2	0.333333	1	3
0.125	0.2	0.2	0.333333	0.2	0.5	0.333333	1

Let's first calculate r_i and r_j ¹²:

```

RI<-(x/A)*(1%8)
RJ<-(x/-A)*(1%8)
RI
4.03308 2.64561 1.52982 1.27987 1.04192 0.501484 0.31178 0.293826
RJ
0.247949 0.377984 0.653672 0.781328 0.959769 1.99408 3.20739 3.40337

```

The calculation of the eigenvector W_i is soon done:

```

WI<-RI% (+/RJ)
WI
0.346916 0.227569 0.131591 0.110091 0.0896231 0.0431364 0.0268185 0.0252742

```

You see how easy is this calculation with APL? It can be automatized in a program, in a very natural way, without complexity. The next calculation returns the eigenvalue $\lambda_{\max}$, which is the average value of all the final vector's values:

¹⁰ <https://www.acquesotteranee.net/acque/article/view/726/534>; the article is in English.

¹¹ This definition is mine. I cannot call these matrices *antisymmetric*, because this is the definition of matrices with negative counterparts. If you know the real name of these matrices, tell me.

¹² See also the theoretical bases on the article *Some Notes on the Similarity of Priority Vectors Derived by the Eigenvalue Method and the Geometric Mean Method* by Jiří Mazurek, Konrad Kułakowski, Sebastian Ernst and Michał Strada, 2022, available for free at sciencedirect.com.

```
LM<-(+/- (A+.xWI) %\oWI) %8
LM
8.91306
```

This is precisely the same result quoted in the article, and since $\lambda_{\max} \geq 8.0$, this is a first assessment that the matrix is good; this leads to the determination of the Consistency Index and the Consistency Ratio:

```
CI<-(LM-8) %7
CI
0.130437
CR<-CI%1.41
CR
0.0925087
```

The Consistency Ratio of 0.0925087 means 9%, that is lesser than 10%, the definitive assessment that the matrix A is acceptable. Just to verify this method, here is the maximum eigenvalue returned by the operator a:

```
} /1 1\oaA
WARNING: THIS EIGENSPACE HAS COMPLEX RESULTS
8.92293
```

The correct value 8.92293 differs slightly from the value 8.91306 of the RGM Method, and this shows that this technique is rather good¹³. This technique is not precise, but so easy to use that hydrogeology (which does not require pharmacist precision) can very well use it profitably.

Can this method be exported to any matrix? Let's try.

```
B
7  3  4  11  9  2
6  4  5  7  1  12
1  9  2  2  9  1
8  0  1  5  0  8
4  3  5  7  2  10
6  1  4  11  7  1
RI<-(x/B) * (1%6)
RJ<-(x/-B) * (1%6)
WI<-RI% (+/RJ)
LM<-(+/- (B+.xWI) %\oWI) %6
LM
20.7585
}/1 1\oaB
WARNING: THIS EIGENSPACE HAS COMPLEX RESULTS
29.0855
```

The first result, with a 6×6 matrix does not return a satisfactory eigenvalue. Another matrix?

```
M
5  4  2  1
0  _5  _1  _1
```

¹³ Matrix A is not invertible, and it has some complex results, but the first and last eigenvalues are real, and the complex ones have absolute values which are lower (this is certified by Wolfram Alpha). So the determination of 8.92293 as the greater eigenvalue (the very same value returned by Wolfram Alpha) is the right one.

```

_1 _1 3 0
1 1 _1 2
RI<-(x/|M)*(1%4)
RJ<-(x/-|M)*(1%4)
WI<-RI% (+/RJ)
LM<-(+/-((|M)+.xWI)%\oWI)%4
LM
2.3969
}/1 1\oaM
4.0004

```

Also in this case the result is not good.

Conclusion: the RGM Method can be used only in case of the peculiar hypo-symmetric matrix analysis in the cited article.

The proof? Another hypo-symmetric matrix.

```

E
1 10 1 11
0.1 1 5 12
1 0.2 1 14
0.0909091 0.0833333 0.0714286 1
RI<-(x/E)*(1%4)
RJ<-(x/-E)*(1%4)
RI
3.23853 1.56508 1.29357 0.152519
RJ
0.308782 0.638943 0.773055 6.55655
WI<-RI% (+/RJ)
LM<-(+/-((E)+.xWI)%\oWI)%4
LM
5.76165
}/1 1\oaE
WARNING: THIS EIGENSPACE HAS COMPLEX RESULTS
5.84093

```

This time, the eigenvalue is quite correct. Only by chance?

8. CHOLESKY DECOMPOSITION

The Cholesky Decomposition, called also *Cholesky factorization*, decomposes a symmetric and positive definite¹⁴ square matrix A into one single lower triangular matrix L so that:

$$A = L \cdot L^T$$

The algorithm is a simple transformation of the matrix A according to the following directives:

$$L_{ij} = \begin{cases} \sqrt{A_{ij} - \sum_{k=1}^{j-1} L_{ik} L_{ik}^T} & i = j \\ \frac{1}{L_{jj}} (A_{ij} - \sum_{k=1}^{j-1} L_{ik} L_{jk}^T) & j < i \\ 0 & j > i \end{cases}$$

These directives cannot apply to a non-symmetric matrix. Since it considers only the lower triangle plus diagonal, to retrieve the Cholesky matrix, if it should be applied to a non-symmetric matrix when used to rebuild A , the result matrix would not be A , but a symmetric matrix around the values of the lower triangle plus diagonal of A (which is its duty), therefore causing a wrong rebuild.

These directives cannot apply to a non-positive definite matrix either (that is, some diagonal elements are negative), or the rebuild fails in the diagonal positions where there is a negative value.

The function `lA` (or `l [1] A`) returns the matrix L , which can be used by post multiplication with its transpose to return the original matrix A . As an example, let's use it with the test matrix `BS`:

BS

280	_160	_131	_131	_154	247
_160	271	_5	184	232	_148
_131	_5	172	24	9	_93
_131	184	24	154	152	_115
_154	232	9	152	203	_142
247	_148	_93	_115	_142	224

Is it invertible?

dBS

1.33956E+07

It is. Now let's decompose it:

lBS

16.7332	0	0	0	0	0
_9.56183	13.4004	0	0	0	0
_7.82875	_5.9593	8.67165	0	0	0
_7.82875	8.14473	1.29705	4.96908	0	0
_9.20326	10.7459	0.113956	_1.5537	0.630932	0
14.7611	_0.511711	2.25001	0.364269	_0.541468	0.600376

¹⁴ https://math.emory.edu/~lchen41/teaching/2020_Fall/Section_8-3.pdf

```
(lBS)+.x\o1BS

280  _160  _131  _131  _154  247
_160  271    _5  184   232  _148
_131    _5  172   24    9   _93
_131  184   24  154   152  _115
_154  232    9  152   203  _142
247  _148  _93  _115  _142  224
```

Another example involves the matrix DS:

```
DS

5  0  0  0
0  1  0  0
0  0  0  0
0  0  0  1
dDS
0
```

This is not invertible. But does the Cholesky Decomposition work?

```
lDS

2.23607      0      0      0
      0      1      0      0
      0      0      0      0
      0      0      0      1
(lDS)+.x\o1DS
```

```
5  0  0  0
0  1  0  0
0  0  0  0
0  0  0  1
```

Maybe you are not convinced by this example, which is a simple diagonal matrix. Here's another more challenging:

```
W

5  4  5  4
4 29  4 29
5  4  5  4
4 29  4 29
dW
0

lW

2.23607      0      0      0
1.78885    5.07937      0      0
2.23607      0  2.98023E_08      0
1.78885    5.07937      0      0
(lW)+.x\o1W

5  4  5  4
```

```

4  29  4  29
5   4  5   4
4  29  4  29

```

This definitely demonstrates that the inversion is not a condition for the Cholesky Decomposition.

In special cases where the matrices change incredibly, the algorithm keeps working; look at the following matrix:

```

D3
      36      720    _3024
      720    14400   _60480
 _3024  _60480  254016
  1D3
      6      0      0
     120     0      0
    _504     0      0

```

The reduction is evident, and the matrix is reduced to a simple vertical vector in form of a matrix. But the surprise is that the rebuild is plain:

```

(1D3)+.x\o1D3
      36      720    _3024
      720    14400   _60480
 _3024  _60480  254016

```

8.1. Applying the Cholesky Decomposition

The Cholesky Decomposition has a surprisingly large number of applications, that we are going to examine, with some help from Wikipedia.

Linear equations. The main use of this kind of decomposition is in solving linear equations of the type $A \cdot x = b$, in all cases where A is symmetric. We can thus assume that in case of this equation:

$$A \cdot x = b$$

with A symmetric, we can assume the following decomposition:

$$A = L \cdot L^T$$

which, applied to the main equation, gives:

$$L \cdot L^T \cdot x = b$$

We can now define a substitution variable y :

$$y = L^T \cdot x$$

so that the equation becomes:

$$L \cdot y = b$$

The last is a simple equation system with a lower triangular matrix, which can be solved by a fast back substitution for finding y ; now that y is known, we can solve the main equation:

$$L^T \cdot x = y$$

by forward substitution, with an upper triangular matrix, for finding x , and solve the system.

You may wonder why using this approach, when the classic method involving the inversion of A may be more direct? Well, my answers are:

- 1 First of all, nothing guarantees that A is invertible. The conditions required for the Cholesky Decomposition is that the matrix must be square and symmetric (Hermitian, in the classic generalization for complex numbers), but nothing requires it must be invertible.
- 2 You should consider that when Mr. Cholesky developed this method, the computer were far to be conceived yet, and an approach *by hand* was the standard; in this regard, the Cholesky Decomposition of A is surely easier to be calculated than the inversion of A , and when L is known, the remaining jobs being the solving of the equations by back and forward substitution, something that is easier manually.
- 3 In any case, even with modern computer approach, the inversion of a matrix can be computationally more challenging than the construction of the Cholesky Decomposition, so even today this method can be preferable.

Example:

```

E
14      43      18      78
43      165     283     219
18      283     2954    306
78      219     306     563

I
1  0  0  0
0  1  0  0
0  0  1  0
0  0  0  1
B<-2 5 6 9
Y<-B| % | 1E
Y
0.534522 _0.199162 0.307718 _1.1243
X<-Y| % | (\o1E)
X
3.1814 _0.618584 0.0646219 _0.219279

```

This is the correct result, because the direct solution is:

```

B| % | E
3.1814 _0.618584 0.0646219 _0.219279

```

For linear systems that can be put into symmetric form, the Cholesky decomposition (or its LDL variant) is the method of choice, for superior efficiency and numerical stability. Compared to the LU Decomposition, it is roughly twice as efficient¹⁵.

¹⁵ Wikipedia at https://en.wikipedia.org/wiki/Cholesky_decomposition.

Inversions of symmetric matrices. Even the matrix inversion can be a subject of the Cholesky Decomposition: in case of symmetric matrices, the mathematical sequence is this:

$$\begin{aligned} A \cdot A^{-1} &= I \\ A &= L \cdot L^T \\ L \cdot L^T \cdot A^{-1} &= I \end{aligned}$$

At first, posing $u = L^T \cdot A^{-1}$, we solve:

$$L \cdot u = I$$

At this point u is a known matrix. Now we can solve for:

$$\begin{aligned} L^T \cdot A^{-1} &= u \\ (L^T)^{-1} \cdot L^T \cdot A^{-1} &= (L^T)^{-1} \cdot u \\ A^{-1} &= (L^T)^{-1} \cdot u \end{aligned}$$

The inverse is thus found. Let's find an example with matrix E, that in APL is easily solvable through the inversion operator `| % |` not applied to E (too easy!); the example stores in L the Cholesky Decomposition of E and in U the first equation result:

```

E
14      43      18      78
43     165     283     219
18     283    2954     306
78     219     306     563
I
1  0  0  0
0  1  0  0
0  0  1  0
0  0  0  1
L<-1E
U<-I | % | L
U
0.267261      0      0      0
_0.535247    0.174266      0      0
0.541864   _0.187788    0.0271551      0
_2.4595     0.467929   _0.0500454    0.195037
( | % | \oL ) + .xU
6.70067      _1.3459      0.137801    _0.479692
_1.3459      0.284591    _0.0285171    0.0912633
0.137801    _0.0285171    0.00324194   _0.00976068
_0.479692    0.0912633   _0.00976068    0.0380393
| % | E
6.70067      _1.3459      0.137801    _0.479692
_1.3459      0.284591    _0.0285171    0.0912633
0.137801    _0.0285171    0.00324194   _0.00976068
_0.479692    0.0912633   _0.00976068    0.0380393

```

QED.

Inversions of non-symmetric matrices. Even in case of non-symmetric cases, the inversion is possible through:

$$B^{-1} = B^T (B \cdot B^T)^{-1}$$

As an example, consider the following matrix B:

B					
7	3	4	_11	_9	_2
_6	4	_5	7	1	12
_1	_9	2	2	9	1
_8	0	_1	5	0	8
_4	3	_5	7	2	10
6	1	4	_11	_7	_1
(\oB) + .x % (B+ .x \oB)					
0.369672	_0.341803	0.0860656	0.00819672	0.440164	_0.287705
0.616393	1.39016	0.196721	_0.409836	_1.3082	_0.714754
1.31667	1.11667	0.416667	_0.166667	_1.11667	_1.31667
0.583333	_0.416667	0.0833333	0.166667	0.416667	_0.583333
0.216393	1.19016	0.196721	_0.409836	_1.1082	_0.314754
0.169672	0.0581967	0.0860656	0.00819672	0.0401639	_0.0877049
% B					
0.369672	_0.341803	0.0860656	0.00819672	0.440164	_0.287705
0.616393	1.39016	0.196721	_0.409836	_1.3082	_0.714754
1.31667	1.11667	0.416667	_0.166667	_1.11667	_1.31667
0.583333	_0.416667	0.0833333	0.166667	0.416667	_0.583333
0.216393	1.19016	0.196721	_0.409836	_1.1082	_0.314754
0.169672	0.0581967	0.0860656	0.00819672	0.0401639	_0.0877049

Of course, the matrix $B \cdot B^T$ is symmetric, and can be decomposed by the Cholesky Decomposition; why is the Cholesky Decomposition useful here? Look at the following development, where L identifies the triangular decomposition of $B \cdot B^T$:

$$B^{-1} = B^T (B \cdot B^T)^{-1}$$

$$B^{-1} = B^T (L \cdot L^T)^{-1}$$

$$B^{-1} = B^T \cdot (L^T)^{-1} L^{-1}$$

Let's prove this conclusion; first of all let's fix in L1 the inverse of L and in L2 the inverse of the transpose of L :

$$L1 <- | \% | 1BB$$

$$L2 <- | \% | \o 1BB$$

Now let's prove the final equation:

(\oB) + .x L2 + .x L1					
0.369672	_0.341803	0.0860656	0.00819672	0.440164	_0.287705
0.616393	1.39016	0.196721	_0.409836	_1.3082	_0.714754
1.31667	1.11667	0.416667	_0.166667	_1.11667	_1.31667
0.583333	_0.416667	0.0833333	0.166667	0.416667	_0.583333

0.216393	1.19016	0.196721	_0.409836	_1.1082	_0.314754
0.169672	0.0581967	0.0860656	0.00819672	0.0401639	_0.0877049
% B					
0.369672	_0.341803	0.0860656	0.00819672	0.440164	_0.287705
0.616393	1.39016	0.196721	_0.409836	_1.3082	_0.714754
1.31667	1.11667	0.416667	_0.166667	_1.11667	_1.31667
0.583333	_0.416667	0.0833333	0.166667	0.416667	_0.583333
0.216393	1.19016	0.196721	_0.409836	_1.1082	_0.314754
0.169672	0.0581967	0.0860656	0.00819672	0.0401639	_0.0877049

QED.

Monte Carlo Simulation. Let's assume we have a variance-covariance matrix Σ (a square matrix giving the covariance between each pair of elements of a given random vector) of dimension $N \times N$ and a vector ξ of random values that have the mean μ_N . Suppose that we want to simulate the Monte Carlo method and try to transform ξ into a multivariate normal vector x , the method is the following: decompose Σ in its Cholesky decomposition form $\Sigma = L \cdot L^T$, then the vector we are looking for is:

$$x = \mu_N + L \cdot \xi$$

The mathematical justification is that it follows directly from the Cholesky decomposition; if the covariance function is expressed by $E()$, it descends that:

$$E[(x - E(x))^T \cdot (x - E(x))] = E[(L \cdot \xi)^T \cdot (L \cdot \xi)] = L^T \cdot E[\xi^T \xi] \cdot L = L^T \cdot I_N \cdot L = \Sigma$$

Let's verify this theory with an example:

```

EPS<-4p4?40
EPS
19  22  4  29
MEAN<- (+/EPS) %4
MEAN
18.5
SIGMA
      1  0.7777  0.39  0.472
0.7777      1  0.88  0.92
0.39      0.88      1  0.95
0.472      0.92  0.95      1
L<-LSIGMA
L
      1      0      0      0
0.7777  0.628636      0      0
0.39    0.917379  0.0794735      0
0.472   0.879565 _0.515567  0.512081
X<-MEAN+L+.xEPS
X
37.5
47.1063
46.4102
59.6065

```

The result is the desired vector; in other words, if we want to find what matrix we have to multiply ξ with, to create a distribution with the covariance matrix Σ , this matrix is L .

9. LDL DECOMPOSITION

The LDL decomposition decomposes a symmetric and positive definite matrix A in the lower triangular matrix L and a diagonal matrix D , so that:

$$A = L \cdot D \cdot L^T$$

It's sometimes classified as a Cholesky Decomposition variant, because the object is the same (symmetric and positive definite), but also because the result variables have these properties:

- The matrix L is the same of the Cholesky Decomposition, where each column is divided by the topmost non-null value; its diagonal is therefore made of ones because, since the topmost non-value lies in the diagonal, each diagonal value is divided by itself.
- The matrix D has in the diagonal the square values of the correspondent values in the diagonal of the Cholesky Decomposition.

This means that to retrieve the LDL Decomposition it needs to retrieve the Cholesky Decomposition first. That's why in `apple` the operator `1` returns also the LDL Decomposition: `1 [2]` will return the matrix L and `1 [3]` the diagonal matrix D .

Examples:

```

J
  4   12  _16
 12   37  _43
_16  _43   98
1 [2] J

  1   0   0
  3   1   0
_4   5   1
1 [3] J

  4   0   0
  0   1   0
  0   0   9
(1 [2] J) + .x (1 [3] J) + .x \o 1 [2] J

  4   12  _16
 12   37  _43
_16  _43   98

```

Again here, even where the transformation change matrices incredibly, the algorithm works; look at the following matrix (the same in the Cholesky case):

```

D3

  36      720  _3024
 720   14400  _60480
_3024  _60480  254016
1 [2] D3

  1   0   0

```

$$\begin{array}{ccc} 20 & 0 & 0 \\ -84 & 0 & 0 \\ 1[3]D3 \end{array}$$

$$\begin{array}{ccc} 36 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{array}$$

The matrix is transformed heavily: the lower triangular matrix L is a vertical vector, and the diagonal matrix is a single value! Nonetheless, the matrix is rebuilt:

$$(1[2]D3) + .x(1[3]D3) + .x \circ 1[2]D3$$

$$\begin{array}{ccc} 36 & 720 & -3024 \\ 720 & 14400 & -60480 \\ -3024 & -60480 & 254016 \end{array}$$

I'm always very impressed by the performance of these matrices.

9.1. Applying the LDL Decomposition

As in the case of the Cholesky Decomposition, the usefulness of the LDL Decomposition is found where the inversion of the single L and D matrices is more convenient than the inversion of A , as it's shown in the following:

$$\begin{aligned} A \cdot x &= b \\ L \cdot D \cdot L^T \cdot x &= b \\ L^{-1} \cdot L \cdot D \cdot L^T \cdot x &= L^{-1} \cdot b \\ D \cdot L^T \cdot x &= L^{-1} \cdot b \\ D^{-1} \cdot D \cdot L^T \cdot x &= D^{-1} \cdot L^{-1} \cdot b \\ L^T \cdot x &= D^{-1} \cdot L^{-1} \cdot b \\ (L^T)^{-1} \cdot L^T \cdot x &= (L^T)^{-1} \cdot D^{-1} \cdot L^{-1} \cdot b \\ x &= (L^T)^{-1} \cdot D^{-1} \cdot L^{-1} \cdot b \end{aligned}$$

The solution is expressed at the cost of the inversion of two triangular matrices (which is fairly easier than a full rank matrix) and one diagonal, which is very simple. You should not ignore these facts, because in certain physical calculus, where the physical phenomenon has symmetric behaviour, and matrix A is symmetric, but has millions of values, requiring its inversion may put the computer at risk, while inverting a triangular matrix is faster.

There is also another solution, by using multiple substitutions in series:

$$\begin{aligned} A \cdot x &= b \\ L \cdot D \cdot L^T \cdot x &= b \\ \text{by posing } y &= L^T \cdot x \\ L \cdot D \cdot y &= b \\ \text{by posing } z &= D \cdot y \\ L \cdot z &= b \end{aligned}$$

Now the solving of z is a matter of back-substitutions:

$$z = L^{-1} \cdot b$$

Having found z , y is easy (it's a diagonal system):

$$y = D^{-1} \cdot z$$

Once y is in the vault, x is directly determinable:

$$x = (L^T)^{-1} \cdot y$$

The examples will use the following data, also in the Cholesky Decomposition case:

```

E
14      43      18      78
43     165     283     219
18     283    2954     306
78     219     306     563
B
2  5  6  9

```

The first example will apply the substitution $A = L \cdot D \cdot L^T$, that ultimately yields $x = (L^T)^{-1} \cdot D^{-1} \cdot L^{-1} \cdot b$:

```

X<-(| % |\o1[2]E)+.x(| % |1[3]E)+.x(| % |1[2]E)+.xB
X
3.1814
_0.618584
0.0646219
_0.219279

```

The second example applies the multiple substitutions:

```

Z<-B| % |1[2]E
Z
2  _1.14286  11.3319  _5.76454
Y<-Z| % |1[3]E
Y
0.142857  _0.0347072  0.0083561  _0.219279
X<-Y| % |(\o1[2]E)
X
3.1814  _0.618584  0.0646219  _0.219279

```

The results are the same of the application of the Cholesky Decomposition, even if the first is a vertical vector that is the *real* result of the calculation.

10. LU/PLU DECOMPOSITION

The LU Decomposition transforms a square matrix A into two special matrices L (lower triangular) and U (upper triangular); the recombination is possible through:

$$A = L \cdot U$$

The whole process is equivalent to the Gaussian elimination, because it corresponds to a set of $n - 1$ matrix operations (being n the order of the system) represented by

$$A^{n-1} = L \cdot U$$

where

$$L = L^{(n-1)} \cdot L^{(n-2)} \cdot \dots \cdot L^{(1)}$$

with each $L^{(r)}$, for $r = 1, 2, \dots, n - 1$, completing one step of the elimination. See also Tao Pang, *Computational Physics*, 2nd edition, Cambridge 2006, page 132.

In case some sparse values cause the appearing of some null values on the diagonal of A , its rows are reordered. This row-reordering is recorded into a special unitary matrix P , that becomes necessary at this point to rebuild A ; this decomposition is called PLU decomposition:

$$P \cdot A = L \cdot U$$

The three matrices P , L and U have many interesting properties:

- The matrix P is equal to its inverse and transpose, $P = P^{-1} = P^T$, so that $P \cdot P^{-1} = P^{-1} \cdot P = P \cdot P^T = P^T \cdot P = P \cdot P = I$.
- The matrix L , built according to the Doolittle convention, has all signed ones on the main diagonal; therefore, its determinant is always 1 or -1.
- The determinant of the matrix U , but its sign, is the multiplication of all the items on the main diagonal.
- For what in the two previous points, the determinant of the original matrix is the multiplication of the determinant of L and the determinant of U , that is the multiplications of all the collected diagonal elements of L and U .
- The inverse of the matrix L is a lower triangular inverse, and is always possible.
- The matrix U has the same echelon reduction of the original matrix A , and therefore is invertible if A is invertible, and non-invertible if A is not invertible (in this case, the rank is smaller than the order of the matrix, and since this means it has some null rows, the determinant is null because some elements of the main diagonal are null; hence the non-invertibility).
- The echelon reduction of the matrix L is the identity matrix.

The above properties can be used extensively, and in particular, the matrix P properties enables the following:

$$\begin{aligned} P \cdot A &= L \cdot U \\ P \cdot P \cdot A &= P \cdot L \cdot U \\ A &= P \cdot L \cdot U \end{aligned}$$

The last expression justifies the name PLU Decomposition.

The operator `l` (the same for the other triangular decompositions) was extended to host even the LU/PLU Decompositions, in this way: `l [ 4 ]` returns the lower triangular matrix L , `l [ 5 ]` returns the upper triangular matrix U , `l [ 6 ]` returns the permutation matrix P .

Let's see some examples; the first case is the case of an invertible matrix:

```
MM
16      2      3      4      5
 6      7      8      9     10
11     12     13     14     15
16     17     18    _16     20
21     22     23     24    _25
 1[4]MM
      1      0      0      0      0
0.375      1      0      0      0
0.6875     1.7      1      0      0
      1     2.4      2      1      0
1.3125     3.1      3      0      1
 1[5]MM
16      2      3      4      5
 0     6.25  6.875    7.5   8.125
 0      0   _0.75   _1.5   _2.25
 0      0      0   _35      0
 0      0      0      0   _50
(1[4]MM)+.x1[5]MM
16      2      3      4      5
 6      7      8      9     10
11     12     13     14     15
16     17     18    _16     20
21     22     23     24    _25
```

The recombination returned the original matrix MM. The second example uses a non-invertible matrix:

```
M
 1      2      3      4      5
 6      7      8      9     10
11     12     13     14      1
 2      3      4      5      6
 7      8      9     10     11
 1[4]M
      1      0      0      0      0
 6      1      0      0      0
11      2      1      0      0
 2     0.2      1      1      0
 7     1.2      1      1      1
 1[5]M
      1      2      3      4      5
 0     _5    _10    _15    _20
 0      0      0      0    _14
 0      0      0      0     14
 0      0      0      0      0
```

(1[4]M)+.x1[5]M

1	2	3	4	5
6	7	8	9	10
11	12	13	14	1
2	3	4	5	6
7	8	9	10	11

Again, the recombination returned the original matrix. Notice that three values in U are null. This descends from the fact the matrix M is not invertible.

In case of sparse matrices, that typically occur with some nulls on the diagonal, the algorithm requires some divisions by zero. The row reordering takes places but anyway, if on the diagonal there are still some null values, the algorithm activates the)DIV 1 option (without really activating the relative flag), and process divisions by zero according to these properties:

- Any number different by zero, divided by zero, returns zero;
- Zero divided zero returns one.

Here's a living demonstration:

R

0	0	0	0	0
0	0	1	0	0
0	1	1	1	0
0	1	1	0	0
0	0	0	0	0

1[4]R

1	0	0	0	0
1	1	0	0	0
1	0	1	0	0
1	1	0	1	0
1	0	0	0	1

1[5]R

0	0	0	0	0
0	1	1	1	0
0	0	1	0	0
0	0	0	-1	0
0	0	0	0	0

1[6]R

0	0	0	0	1
0	0	1	0	0
0	1	0	0	0
0	0	0	1	0
1	0	0	0	0

In this case, the rebuild needs P , which was rearranged (it's not the identity matrix):

(1[6]R)+.x(1[4]R)+.x(1[5]R)

```

0 0 0 0 0
0 0 1 0 0
0 1 1 1 0
0 1 1 0 0
0 0 0 0 0

```

Another special case is the zero-diagonal case:

```

      H
      0   2   3   4   5
      6   0   8   9  10
    11  12   0  14   1
      2   3   4   0   6
      7   8   9  10   0
    1[4]H
          1           0           0           0           0
          0           1           0           0           0
    0.333333          1.5           1           0           0
    1.833333          6    10.3158           1           0
    1.16667           4    3.89474   0.279651           1
    1[5]H
          6           0           8           9           10
          0           2           3           4           5
          0           0   _3.16667           _9   _4.83333
          0           0           0    66.3421    2.52632
          0           0           0           0   _13.5486
    1[6]H
    0  1  0  0  0
    1  0  0  0  0
    0  0  0  1  0
    0  0  1  0  0
    0  0  0  0  1
    (1[6]H)+.x(1[4]H)+.x(1[5]H)
    0   2   3   4   5
    6   0   8   9  10
    11  12   0  14   1
    2   3   4   0   6
    7   8   9  10   0

```

Even in this weird case, the rebuild is perfect.

10.1. Applying the LU/PLU Decomposition

As the Cholesky Decomposition, the LU/PLU decomposition can be used to solve many problems. Let's see some of them.

Linear equations. The LU/PLU Decompositions are useful in solving linear equations of the type:

$$A \cdot x = b$$

The process of substitution is:

$$L \cdot U \cdot x = b$$

In case there are permutations, the system becomes:

$$P \cdot L \cdot U \cdot x = b$$

$$P \cdot P \cdot L \cdot U \cdot x = P \cdot b$$

$$L \cdot U \cdot x = P \cdot b$$

The last equation uses the property of $P = P^{-1} = P^T$. By a first substitution we define

$$U \cdot x = y$$

so that our equation becomes:

$$L \cdot y = P \cdot b$$

This is a solvable system in y ; after solving, now that y is known, we can solve for x :

$$U \cdot x = y$$

The solutions of the last two equations are easy, because both L and U are triangular.

Let's see an example:

```

E
14      43      18      78
43     165     283     219
18     283    2954     306
78     219     306     563
L<-1 [4]E
U<-1 [5]E
1 [6]E

1  0  0  0
0  1  0  0
0  0  1  0
0  0  0  1

```

The last check assures that there was no reordering of the rows of E , so P can be safely omitted from the calculation:

```

B<-2 5 6 9
Y<-B | % | L
Y
2  _1.14286  11.3319  _5.76454
X<-Y | % | U
X
3.1814  _0.618584  0.0646219  _0.219279

```

The result are the same of the application of the Cholesky Decomposition.

Matrix inversion. Even the matrix inversion can be a subject of the LU/PLU Decomposition, as show next; let's

suppose we are looking for the inverse X of matrix A ; it descends that:

$$A \cdot X = I$$

Applying the decomposition, we derive:

$$\begin{aligned} P \cdot L \cdot U \cdot X &= I \\ L \cdot U \cdot X &= P \cdot I = P \\ L^{-1} L \cdot U \cdot X &= L^{-1} \cdot P \\ U \cdot X &= L^{-1} \cdot P \\ U^{-1} \cdot U \cdot X &= U^{-1} \cdot L^{-1} \cdot P \\ X &= U^{-1} \cdot L^{-1} \cdot P \end{aligned}$$

The problem simply translates to the inversion of two triangular matrices L and U (and possibly some reordering with P).

Example with an invertible matrix:

```

M
  5   4   2   1
  0   1  _1  _1
 _1  _1   3   0
  1   1  _1   2
      L<-1[4]M
      U<-1[5]M
      1[6]M

  1   0   0   0
  0   1   0   0
  0   0   1   0
  0   0   0   1
      ( | % | U) + .x ( | % | L)

      0.1875      _0.8125      _0.5625      _0.5
 _1.38778E_17      1          0.5          0.5
      0.0625      0.0625      0.3125  _8.67362E_18
 _0.0625      _0.0625      0.1875      0.5
      | % | M

      0.1875  _0.8125  _0.5625  _0.5
      0          1      0.5      0.5
      0.0625  0.0625  0.3125      0
 _0.0625  _0.0625  0.1875      0.5

```

The result is astonishing, because the error is very very small.

What if we try to solve a non-invertible matrix?

```

D
  1   0   1   1
  0   4   0   0
  0   0   0   0

```

```

0  4  0  1
  dD
0

```

This matrix is not invertible, because its determinant is null.

```

L<-1[4]D
U<-1[5]D
L

```

```

1  0  0  0
0  1  0  0
0  1  1  0
0  0  1  1
  U

```

```

1  0  1  1
0  4  0  0
0  0  0  1
0  0  0  _1
  1[6]D

```

```

1  0  0  0
0  1  0  0
0  0  0  1
0  0  1  0

```

This time we cannot omit P because it's not the identity matrix.

```

P<-1[6]D
( | % | U) + .x ( | % | L) + .xP
0.5  0.5  0.25  _0.5
  0  0.25    0    0
0.5  0.5  0.25  _0.5
  0  _1  _0.5    1

```

The found result is one pseudo-inverse of A :

```

DI<-( | % | U) + .x ( | % | L) + .xP
DI
0.5  0.5  0.25  _0.5
  0  0.25    0    0
0.5  0.5  0.25  _0.5
  0  _1  _0.5    1
  D+.xDI+.xD
1  0  1  1
0  4  0  0
0  0  0  0
0  4  0  1

```

This double rebuild applies the weak conditions of the Moore-Penrose theory about the pseudo-inverse; the result

returned by the inversion operator (that reaches the result by means of the Ben-Israel iteration) is different but equally valid (because the third line in D is null and it nullifies any third column in the inverse...):

```

| % | D
0.5  0.5  0  _0.5
0  0.25  0  0
0.5  0.5  0  _0.5
0  _1  0  1
I_36
PSEUDO-INVERSE
D+.x ( | % | D ) +.xD

1  0  1  1
0  4  0  0
0  0  0  0
0  4  0  1

```

This shows the power of the LU/PLU decompositions!

Calculation of the determinant. By using the properties of L and U we can calculate the determinant using APL in a very natural way:

```

E
14  43  18  78
43  165 283 219
18  283 2954 306
78  219 306 563
L<-1[4]E
U<-1[5]E
x/( (1 1\oL),1 1\oU)
16434916
dE
16434916
M
5  4  2  1
0  1  _1  _1
_1  _1  3  0
1  1  _1  2
L<-1[4]M
U<-1[5]M
x/( (1 1\oL),1 1\oU)
32
dM
32

```


11. LDU DECOMPOSITION

The LDU Decomposition transforms a square matrix A into three matrices L (unitary lower triangular), D (diagonal) and U (unitary upper triangular); the recombination is possible through:

$$A = L \cdot D \cdot U$$

The algorithm is the very same of the LU decomposition (without reordering), and with the following actions:

- The diagonal matrix D is composed of the diagonal elements of U . The algorithm used, actually, returns a unitary matrix L and a normal upper triangular matrix U , not unitary.
- The matrix U is turned to unitary, dividing each row i by u_{ii}

The operator `l` (the same for the other triangular decompositions) was extended to host even the LDU Decompositions, in this way: `l [7]` returns the unitary lower triangular matrix L , `l [8]` returns the diagonal matrix D , `l [9]` returns the unitary upper triangular matrix U .

Example:

```

M
5   4   2   1
0   1  _1  _1
_1  _1   3   0
1   1  _1   2

L<-l [7]M
D<-l [8]M
U<-l [9]M

L
1   0   0   0
0   1   0   0
_0.2 _0.2 1   0
0.2  0.2 _0.375 1

U
1  0.8 0.4 0.2
0   1  _1  _1
0   0   1   0
0   0   0   1

D
5   0   0   0
0   1   0   0
0   0  3.2   0
0   0   0   2

L+.xD+.xU

5   4   2   1
0   1  _1  _1
_1  _1   3   0
1   1  _1   2

```

Note that there is no permutation matrix here. In case of sparse matrices, that typically occur with some nulls on the

diagonal, the algorithm may require some divisions by zero. The algorithm thus activates the `)DIV 1` option (without really activating the relative flag), and process divisions by zero according to these properties:

- Any number different by zero, divided by zero, returns zero;
- Zero divided zero returns one.
- In the diagonal matrix, all zeroes in u_{ii} are changed to 1.0.

Here's a living demonstration:

```

C
0  2  0  _1
1  0  0  0
0  1  0  0
0  0  1  0
L<-1[7]C
D<-1[8]C
U<-1[9]C
L
1  0  0  0
0  1  0  0
1  0  1  0
1  0  0  1
D
1  0  0  0
0  1  0  0
0  0  1  0
0  0  0  1
U
0  2  0  _1
1  0  0  0
0  _1  0  1
0  _2  1  1
L+.xD+.xU
0  2  0  _1
1  0  0  0
0  1  0  0
0  0  1  0

```

As you can see, the matrix U has zeroes on the diagonal (but the last), and the diagonal matrix D , whereas U reports zero, puts one. This makes D more *transparent* to the execution of the rebuild: maintaining its original zeroes, would returns a quasi-zero matrix.

Another peculiar matrix:

```

D3
36      720    _3024
720    14400   _60480

```

```

_3024 _60480 254016
L<-1[7]D3
D<-1[8]D3
U<-1[9]D3
L
1      0      0
20     1      0
_84    1      1
U
1      20    _84
0      0      0
0      0      0
D
36     0      0
0      0      0
0      0      0
L+.xD+.xU
36      720    _3024
720     14400  _60480
_3024   _60480 254016

```

11.1. Applying the LDU Decomposition

The LDU Decomposition is a base of two other common triangular decompositions¹⁶: the Doolittle Decomposition and the Crout Decomposition.

the Doolittle Decomposition retains the unitary lower triangular L :

$$A = L \cdot U'$$

with the position:

$$U' = D \cdot U$$

The Crout Decomposition, instead, retains the unitary upper triangular U :

$$A = L' \cdot U$$

with the position:

$$L' = L \cdot D$$

The operator `1` returns the L , D and U matrices, and the recombination is a simple matter of matrix multiplication.

¹⁶ See also https://en.wikipedia.org/wiki/Crout_matrix_decomposition

12. SV DECOMPOSITION

The Singular Value Decomposition (SVD) is one of the most important decompositions; according to Wikipedia¹⁷ it's "a factorization of a real or complex matrix into a rotation, followed by a rescaling followed by another rotation. It generalizes the eigen-decomposition of a square normal matrix with an orthonormal eigen-basis to any $m \times n$ matrix. It's related to the Polar Decomposition. [...] Mathematical applications of the SVD include computing the pseudo-inverse, matrix approximation, and determining the rank, range, and null space of a matrix. The SVD is also extremely useful in all areas of science, engineering, and statistics, such as signal processing, least squares fitting of data, and process control."

Carla D. Martin and Mason A. Porter, in their article "The Extraordinary SVD"¹⁸ say: "We find the SVD in statistics in the guise of "principal component analysis" (PCA), which entails computing the SVD of a data set after centering the data for each attribute around the mean. Many other methods of multivariate analysis, such as factor and cluster analysis, have also proven to be invaluable. The SVD per se has been used in chemical physics to obtain approximate solutions to the coupled-cluster equations, which provide one of the most popular tools used for electronic structure calculations. Additionally, we apply an SVD when diagonalizing the one-particle reduced density matrix to obtain the natural orbitals (i.e., the singular vectors) and their occupation numbers (i.e., the singular values). The SVD has also been used in numerous image-processing applications, such as in the calculation of Eigenfaces to provide an efficient representation of facial images in face recognition. It's also important for theoretical endeavors, such as path-following methods for computing curves of equilibria in dynamical systems. The SVD has also been applied in genomics, textual database searching, robotics, financial mathematics, compressed sensing, and more.". It's a peculiar algorithm, "discovered over 100 years ago independently by Eugenio Beltrami (1835-1899) and Camille Jordan (1838-1921). James Joseph Sylvester (1814-1897), Erhard Schmidt (1876-1959), and Hermann Weyl (1885-1955) also discovered the SVD using different methods"¹⁹.

The decomposition of an $m \times n$ matrix A is performed as follows:

$$A = U \cdot \Sigma \cdot V^T$$

in which, if A is $m \times n$, then U , whose columns are called the left-singular vectors, is $m \times m$, V , whose columns are called the right-singular vectors, is $n \times n$ and Σ is $m \times n$ with the singular values on the main diagonal (that is any configuration starting from position 1,1 and going on down-right). In case there are null singular values, some columns of U and V may be null, as some columns and rows in Σ . In this case, if r is the rank of the matrix (that is the number of linearly independent columns or rows), the compact representation of the return values presents U as an $m \times r$ matrix, V as an $n \times r$ matrix and Σ as an $r \times r$ square matrix. The matrices U and V are orthogonal only if $m = n = r$ (that is A is square and invertible); in case of non-square matrices or when the rank is lower than any of the dimensions, they may be left- or right-orthogonal, or even they may be not orthogonal at all. From the mathematical point of view, the singular values are to be interpreted as the magnitude of the rotation for all the included axes.

To calculate the SVD, at first it's assumed that $m \geq n$; if it's not, the argument matrix is transposed, and a flag is set. Next, the algorithm proceeds defining a matrix B as:

$$B = A^T \cdot A$$

A QR algorithm is applied on B , which is a symmetric $n \times n$ matrix; the square values of the n eigenvalues of B (of which at least r eigenvalues are not null) are also the singular values of A ; they are gathered in descending order, and Σ is then populated; the eigenvectors, which are not normalized, constitute the $n \times n$ V matrix. `apple` records the number of iterations needed to complete the QR algorithm on V .

The matrix U is obtained by reversing the equation, in matrix terms (remember that V is orthogonal, and that Σ is diagonally invertible, with the condition that in case of null singular values, a zero is used, instead of letting the

¹⁷ https://en.wikipedia.org/wiki/Singular_value_decomposition

¹⁸ You can read it here: <https://people.maths.ox.ac.uk/~porterm/papers/s4.pdf>

¹⁹ Again Carla D. Martin and Mason A. Porter in the cited document.

compiler use the *nan* entity). This assures that for the given V , the correspondent U is calculated by reversing the equation; the reason behind this decision is that the two results matrices V and U obtained from the two QR algorithms, sometimes are affected by sign differences, that make the rebuild impossible, attesting that the SVD was not well calculated.

The reversing sequence for calculating U is:

$$\begin{aligned} A &= U \cdot \Sigma \cdot V^T \\ A \cdot V &= U \cdot \Sigma \cdot V^T \cdot V \\ A \cdot V &= U \cdot \Sigma \\ A \cdot V \cdot \Sigma^{-1} &= U \cdot \Sigma \cdot \Sigma^{-1} \\ A \cdot V \cdot \Sigma^{-1} &= U \end{aligned}$$

At the end, the matrices are reduced to eliminate the zero-columns and the zero-rows (this process makes the returned values compact, and this is called *compact SVD*).

The mathematical justification behind the SV Decomposition algorithm is this:

Let's assume that the matrix M has the following Singular Values Decomposition representation: $M = U \cdot \Sigma \cdot V^T$; it follows that the Singular Value representation of its transpose is $M^T = V \cdot \Sigma^T \cdot U^T$; consequently, remembering that the transpose of a diagonal matrix is the diagonal matrix itself, the following two relations hold:

$$\begin{aligned} M^T \cdot M &= V \cdot \Sigma^T \cdot U^T \cdot U \cdot \Sigma \cdot V^T = V \cdot (\Sigma^T \cdot \Sigma) \cdot V^T = V \cdot \Sigma^2 \cdot V^T \\ M \cdot M^T &= U \cdot \Sigma \cdot V^T \cdot V \cdot \Sigma^T \cdot U^T = U \cdot (\Sigma \cdot \Sigma^T) \cdot U^T = U \cdot \Sigma^2 \cdot U^T \end{aligned}$$

The right-hand side of these expressions is the Jordan representation of the left-hand side matrix, with the following conclusions:

- The columns of V are the eigenvectors of the matrix $M^T \cdot M$
- The columns of U are the eigenvectors of the matrix $M \cdot M^T$
- The non-null elements in Σ are the square roots of the non-null eigenvalues of both $M^T \cdot M$ and $M \cdot M^T$.

It follows that to calculate the singular values of M , the QR algorithm applied to $M^T \cdot M$ returns the matrix V and the QR algorithm applied to $M \cdot M^T$ returns the U matrix (both do not need to be normalized) and the Σ matrix can be derived from either. The advantage of this approach is that being $M^T \cdot M$ and $M \cdot M^T$ are symmetric, and the singular values are therefore all real.

The function `sA` (or `s [1] A`) performs the compact SV Decomposition on the matrix argument, and returns the Σ matrix; `s [2] A` returns the compact matrix U and `s [3] A` returns the compact matrix V .

As an addition, the function `s [4] A` performs the normal SV Decomposition on the matrix argument, and returns the complete Σ matrix (with zero columns or rows if necessary); `s [5] A` returns the full matrix U and `s [6] A` returns the full matrix V .

The function `s [7]` returns a scalar containing the iterations of the QR algorithm for V .

The first example is a 5×5 invertible matrix:

$$M = \begin{bmatrix} -5 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & -3 & 14 & 15 \end{bmatrix}$$

```

16   17   18   19   20
21   22   23   24   _21
dM
_88320
s[2]M

0.042726    0.135745    0.331794    _0.920823    0.147461
0.254998    0.247792    0.231033    _0.0134078    _0.905549
0.303998    0.448502    _0.813262    _0.212177    0.00398516
0.588603    0.483336    0.407858    0.309177    0.397485
0.703052    _0.69671    _0.0937701    _0.106278    _0.0150195
s[3]M

0.46752    _0.0332113    _0.383395    0.780974    _0.153004
0.502367    0.00796211    _0.214447    _0.248993    0.799737
0.454214    _0.180429    0.860863    0.14133    _0.00868517
0.563749    0.0435101    _0.203978    _0.549461    _0.580327
0.0702133    0.982032    0.155977    0.0787416    0.0124578
sM

61.6916      0      0      0      0
0  34.8072      0      0      0
0      0  12.1601      0      0
0      0      0  6.28025      0
0      0      0      0  0.53858
(s[2]M)+.x(sM)+.x\os[3]M

_5    2    3    4    5
6    7    8    9   10
11   12   _3   14   15
16   17   18   19   20
21   22   23   24   _21

```

The matrix is square and invertible (the determinant is not null), so that it's full-rank ($r = m = n$).

The second example is a 7×3 matrix:

```

M

83  88  42
31  53   9
13  73  19
78  43  39
37  91  26
49  79  23
51   2  80
sM

233.788      0      0
0  89.6738      0
0      0  36.9427
s[2]M

0.545561    _0.037546    _0.293795
0.255596      0.18011    _0.119326

```

```

0.288817    0.340277    0.519749
0.388824   _0.322406   _0.509938
0.415315         0.311    0.297342
0.40312     0.188283   _0.0532757
0.259696   _0.783738    0.527276
s[3]M

```

```

0.580235   _0.418124   _0.698928
0.725095    0.656004    0.209513
0.370897   _0.628356    0.683816
s[7]M

```

22

The full matrices are returned by the functions:

```

s[4]M
233.788      0      0
0  89.6738      0
0      0  36.9427
0      0      0
0      0      0
0      0      0
0      0      0
s[5]M
0.545561   _0.037546   _0.293795      0      0      0
0.255596    0.18011    _0.119326      0      0      0
0.288817    0.340277    0.519749      0      0      0
0.388824   _0.322406   _0.509938      0      0      0
0.415315         0.311    0.297342      0      0      0
0.40312     0.188283   _0.0532757      0      0      0
0.259696   _0.783738    0.527276      0      0      0
s[6]M
0.580235   _0.418124   _0.698928
0.725095    0.656004    0.209513
0.370897   _0.628356    0.683816

```

As you can see, the matrix V is the same in both systems, because it is a full matrix. In any case, the QR algorithm required 22 iterations. The rebuilding of the original matrix is easily done in both cases:

```

(s[2]M) + .x (sM) + .x \os [3]M
83  88  42
31  53   9
13  73  19

```

```

78  43  39
37  91  26
49  79  23
51   2  80
      (s[5]M) + .x (s[4]M) + .x\os[6]M

83  88  42
31  53   9
13  73  19
78  43  39
37  91  26
49  79  23
51   2  80

```

The matrix M is perfectly rebuilt and the matrices V , U and Σ are the right ones. The error is this:

```

M- (s[2]M) + .x (sM) + .x\os[3]M

_5.25802E_13    1.7053E_13    2.27374E_13
_2.23821E_13    9.9476E_14    1.03029E_13
_1.56319E_13    1.98952E_13    1.7053E_13
_4.40536E_13    7.10543E_14    1.42109E_13
_3.12639E_13    1.98952E_13    2.0961E_13
_3.2685E_13     1.7053E_13    1.84741E_13
_2.70006E_13    7.81597E_14    1.7053E_13

```

This is perfectly acceptable, for me, because it's an error around 10^{-12} .

If the matrix rank is 1, the matrices U and V reduce to vectors and Σ reduces to a 1×1 matrix, but the reconstruction is still available, as in the following example:

```

L
4  4  4  4
4  4  4  4
4  4  4  4
4  4  4  4
      rL
1  1  1  1
0  0  0  0
0  0  0  0
0  0  0  0
      r[2]L
1

```

The SV Decomposition is:

```

sL
16
      s[2]L
0.5

```



```
0.5
0.5
0.5
    s[3]L
```

```
0.5
0.5
0.5
0.5
```

The rebuild is:

```
(s[2]L)+.x(sL)+.xL
4  4  4  4
4  4  4  4
4  4  4  4
4  4  4  4
```

The general consideration is that the compact form depends on the rank of the original matrix.

Another working example features a 5×3 matrix:

```

M
97  56  24
17  39  60
50  26  68
90   9  53
85  22  58
    sM
205.251      0      0
      0  57.3038      0
      0      0  41.5024
    s[2]M
0.521998  _0.507433  _0.679986
0.278867   0.764354  _0.353609
0.380643   0.361901  0.0593726
0.49552    _0.16502   0.568418
0.509238  _0.00836241  0.293182
    s[3]M
0.790685  _0.587999   0.170512
0.319937   0.159393  _0.933935
0.521974   0.793001   0.314152
    rM
3
    (s[2]M)+.x(sM)+.x\s[3]M
97  56  24
17  39  60
50  26  58
```

```
90  9  53
85 22  58
```

The next examples features the same matrix, transposed:

```
M<-\oM
M

97 17  50  90  85
56 39  26   9  22
24 60  58  53  58
sM

205.251      0      0
      0  57.3038      0
      0      0  41.5024
s[2]M

0.790685 _0.587999  0.170512
0.319937  0.159393 _0.933935
0.521974  0.793001  0.314152
s[3]M

0.521998 _0.507433 _0.679986
0.278867  0.764354 _0.353609
0.380643  0.361901  0.0593726
0.49552   _0.16502  0.568418
0.509238 _0.00836241 0.293182
rM
3
(s[2]M)+.x(sM)+.x\os[3]M

97 17  50  90  85
56 39  26   9  22
24 60  58  53  58
```

The rebuild is perfect, U and V are exchanged, SIGMA transposed and the range is 3 again (of course).

Another small case that I like:

```
N

1  2
3  4
5  6
7  8
sN

14.2691      0
      0  0.626828
s[2]N

0.152483  0.822647
0.349918  0.421375
```

```

0.547354  0.0201031
0.744789  _0.381169
      s[3]N

0.641423  _0.767187
0.767187  0.641423
      rN
2
      (s[2]N)+.x(sN)+.x\os[3]N

1  2
3  4
5  6
7  8

```

The next example is a square non invertible matrix, and the result are in compact form because there are null singular values:

```

      M

1   2   3   4
5   6   7   8
9  10  11  12
13 14  15  16
      rM
2
      sM

38.6227      0
      0  2.07132
      s[2]M

0.134722  0.825742
0.340758  0.428817
0.546793  0.0318923
0.752829  _0.365033
      s[3]M

0.428412  _0.718653
0.474373  _0.273808
0.520333  0.171038
0.566293  0.615884
      (s[2]M)+.x(sM)+.x\os[3]M

1   2   3   4
5   6   7   8
9  10  11  12
13 14  15  16
      s[7]M
8

```

The final case that I present here involves the curious case of the SV Decomposition of the identity matrix:

```
I<-4 4p1 0 0 0 0
```

```

      I
1  0  0  0
0  1  0  0
0  0  1  0
0  0  0  1
      sI
1  0  0  0
0  1  0  0
0  0  1  0
0  0  0  1
      s[2]I
1  0  0  0
0  1  0  0
0  0  1  0
0  0  0  1
      s[3]I
1  0  0  0
0  1  0  0
0  0  1  0
0  0  0  1
      s[7]I
1

```

This is the reign of ones: the QR algorithm was accomplished in one cycle; all matrices are identity matrices, with all ones in the diagonal, and zeros in the remainder! And since the transpose of the identity matrix is the identity matrix itself, the rebuild is a tautology:

```

      (s[2]I)+.x(sI)+.x\os[3]I
1  0  0  0
0  1  0  0
0  0  1  0
0  0  0  1
      I+.xI+.x\oI
1  0  0  0
0  1  0  0
0  0  1  0
0  0  0  1

```

The algorithm, on my modest PC, calculates a 100×100 matrix (10.000 elements!) reconstructions (3 times!) as so:

```

R<-100 100p1 2 3
t+ /+ /R- (s[2]R)+.x(sR)+.x\os[3]R
TIMING: 41.8 s
2.661E_11

```

Of course I could have put the three results in three different variables, and proceed from them, but I wanted to see what happened. And the error is contained, 10^{-10} is the limit. Not bad at all for 10.000 elements in memory without any trick for this huge full matrix!

12.1. Applying the SV Decomposition

There are many cases where the SV Decomposition is useful. Let's analyse some of them.

Calculating the pseudo-inverse. The pseudo-inverse of a matrix can be obtained by the SVD applied to the matrix; if $M = U \cdot \Sigma \cdot V^T$, with the hypothesis that M is not invertible in the Gaussian sense (that is, the determinant is null), then its pseudo-inverse is

$$M^+ = V \cdot \Sigma^{-1} \cdot U^T$$

The inversion of Σ is not a problem, because it's a diagonal matrix; the inversion can be completed with the `| % |` inversion operator, or using the reciprocal `%` operator on Σ , but in this case it's necessary enabling `)DIV 1`, to avoid the error in the reciprocal of zero (look carefully at the example):

```

M<-4 4pi16
M

  1   2   3   4
  5   6   7   8
  9  10  11  12
 13  14  15  16
S<-sM
S

38.6227      0
      0  2.07132
)DIV 1
WAS 0
SI<-%S
SI

0.0258915      0
      0  0.482783
U<-s[2]M
V<-s[3]M
MI<-V+.xSI+.x\oU
MI

 _0.285  _0.145  _0.005  0.135
 _0.1075 _0.0525  0.0025  0.0575
  0.07   0.04   0.01   _0.02
 0.2475  0.1325  0.0175 _0.0975

```

The result found for MI is precisely the result of the inversion according to Moore-Penrose (returned by the `| % |` operator):

```

| % | M

 _0.285  _0.145  _0.005  0.135
 _0.1075 _0.0525  0.0025  0.0575
  0.07   0.04   0.01   _0.02
 0.2475  0.1325  0.0175 _0.0975

```

This is a pseudo-inverse, because the original matrix is returned by the three-matrices multiplication:

```

M+.xMI+.xM

1  2  3  4
5  6  7  8
9 10 11 12
13 14 15 16

```

Algebraic rank of a matrix. Another application of the SVD is related to the visual representation of the range and null space of a matrix. In particular, the right-singular vectors (in U) and the left-singular vectors (in V) correspond to the non-zero singular values of M , that is they span the null space in M .

For example, consider the matrix $M3$:

```

M3

1  2  3
4  5  6
7  8  9
SVD M3
12
U

0.214837  0.887231
0.520587  0.249644
0.826338 _0.387943
V

0.479671 _0.776691
0.572368 _0.0756865
0.665064  0.625318

```

Matrix $M3$ is 3×3 , but the non-null columns in U (and V) are 2; this means that the range of $M3$ is 2. This is exemplified also by $SIGMA$:

```

SIGMA
16.8481  0
0  1.06837

```

$SIGMA$ is a 2×2 matrix.

The rank operator r perform a similar task on its own, returning the echelon matrix with axis 1 and the matrix rank with axis 2, and also the count of the linearly independent rows (axis 3) and columns (axis 4) by reducing the matrix in echelon format:

```

r[2]M3
2
r[3]M3
3
r[4]M3
2
rM3

1  2  3
0  1  2

```

0 0 0

The example shows also that M3 has 3 linear independent columns, but only 2 independent rows, so its rank is 2. This reflects in the compact SVD decomposition.

Low-rank conversion (Theorem of Eckart-Young). If a matrix, say M , is to be approximated with another matrix M' , which must have a specific rank r , lesser than the rank of M , the solution is

$$M' = U \cdot \Sigma' \cdot V^T$$

where Σ' is the same Σ that comes from the SVD decomposition $M = U \cdot \Sigma \cdot V^T$, containing only the r largest singular values (the other singular values are replaced by zero). For example:

```
A
1  2  3
4  5  6
7  8  _9
r[2]A
3
U<-s[2]A
V<-s[3]A
SIGMA<-sA
47
SIGMA
13.9876      0      0
      0  9.44355      0
      0      0  0.408805
```

The SIGMA matrix is 3×3 , but in any case A has rank 3. Now let's create SIGMA2 (the Σ' of the mathematical field), by removing the smaller singular value:

```
SIGMA2<-SIGMA
SIGMA2[3;3]<-0
SIGMA2
13.9876      0      0
      0  9.44355      0
      0      0      0
```

The first version we create should have a rank 1-unity lower than A, and precisely 2, because we have reduced the singular values in SIGMA2 by one unity.

```
MS<-U+.xSIGMA2+.x\oV
MS
1.29025  1.76192  3.01492
3.8764  5.10139  5.99364
7.01492  7.98776  _8.99923
r[2]MS
2
```

Verified. And now let's extend the reduction, removing the next smaller singular value:

```

SIGMA2[2;2]<-0
MS<-U+.xSIGMA2+.x\oV
MS

_0.00954894  _0.0109761  0.0106085
 0.866707    0.996243   _0.962873
 7.3673      8.4684    _8.18475
r[2]MS
1

```

The rank of MS was lowered by another unity. This is the Eckart-Young theorem. If you wonder which is the echelon form of the last MS, here it is:

```

rMS

1    1.14946  _1.11096
0         0         0
0         0         0

```

The previous output used the system command)FUZZ 14.

Finding the eigenspace of a well-posed square matrix. The SV Decomposition can be used also for determining the eigenspace (eigenvalues and eigenvectors) of a well-posed matrix. What is a well-posed matrix? Well, this is a definition, applied to matrices, define those who are square, symmetric, invertible and not singular; this is not sufficient, for using this method; a well-posed matrix has also real non-null eigenvalues, all distinct (with multiplicity equal to one). This position is imperative, because the method suggested here, works only with well-posed square (and real) matrices.

If matrix A has been decomposed as follows, with the hypothesis to avoid in any case the compact form:

$$A = U \cdot \Sigma \cdot V^T$$

the process is coordinated by the following simple expression:

$$P = V \cdot \Sigma^{-0.5}$$

The calculation is always possible, because Σ is diagonal, and thus always invertible (with the hypothesis that in case of division by zero - eigenvalue null case - a zero is returned).

The matrix P holds the eigenvectors of A (one of the possible set of eigenvectors, I should say²⁰), in non-normalized form; each column of P with n elements must follow this operation on row r if the last column item is not zero or one:

$$P[:,r] = P[:,r]/P[n,r], r = 1..n$$

in APL this translates to:

```

F[:,R]<-F[:,R]%F[N,R]

```

Having processed the eigenvectors, the Jordan eigenvalue matrix is a simple matter of matrix multiplications:

²⁰ In general the eigenvectors found through the SV Decomposition (again, one of the possible SV decompositions, I should say) are different from those derived from the QR algorithm, but the eigenvalues should be the same.

$$J = P^{-1} \cdot A \cdot P$$

This shows once more how powerful is the SV Decomposition.

For the following example, I've chosen a well-posed square matrix, with full diagonal; the sequence must use the)DIV 1 command, because we must elevate a diagonal matrix to power -0.5:

```

M
46  1  _3  9
   1  3  _4  0
  _3  _4  11 _5
   9  0  _5  7
    V<-s[3]M
    S<-sM
    V
      0.967904  _0.193333  0.14155  _0.0758168
0.0310346    0.29785  0.594364  0.74636
  _0.11068  _0.858063  _0.160723  0.475021
0.223493    0.370991  _0.775154  0.459949
    S
      48.4533      0      0      0
         0  13.8743      0      0
         0      0  4.3198      0
         0      0      0  0.352616
    )DIV 1
WAS 0
    P<-V+.x(S*_0.5)
    P
      0.13905  _0.051904  0.0681051  _0.127678
0.00445845  0.0799635  0.28597  1.25689
  _0.0159003  _0.230363  _0.0773295  0.799949
0.0321073  0.0995997  _0.372955  0.774567
    P[;1]<-P[;1]%P[4;1]
    P[;2]<-P[;2]%P[4;2]
    P[;3]<-P[;3]%P[4;3]
    P[;4]<-P[;4]%P[4;4]
    P
      4.33079  _0.521126  _0.18261  _0.164837
0.138861  0.802849  _0.76677  1.6227
  _0.495226  _2.31289  0.207343  1.03277
         1         1         1         1

```

This is the eigenvector matrix P ; don't you believe? Look at the real matrix:

```

a[2]M
      4.33079  _0.521126  _0.18261  _0.164837
0.138861  0.802849  _0.76677  1.6227

```

```
_0.495226  _2.31289  0.207343  1.03277
      1      1      1      1
```

The calculation of the eigenvalues stems from the main equation:

$$J < - \left(\frac{1}{\%} \right) P + .xM + .xP$$

```
J
48.4533      0      0  3.14033E_12
      0  13.8743      0      0
      0      0  4.3198      0
      0      0      0  0.352616
```

This is the real matrix:

```
aM
48.4533      0      0      0
      0  13.8743      0      0
      0      0  4.3198      0
      0      0      0  0.352616
```

The results are quite perfect: the error is very poor (less than 10^{-12}); let's quantify it overall:

```
)FUZZ 0
WAS 12
J-aM
_7.10543E_15  _4.66294E_15  _7.27196E_15  3.14033E_12
_5.32907E_15  3.55271E_15  _1.80411E_16  1.16036E_14
_1.06581E_14  _3.33067E_16  _1.77636E_15  _9.88983E_14
_4.44089E_15  1.94289E_15  _1.26635E_15  4.44089E_16
P-a[2]M
_6.21725E_15  _2.22045E_15  2.02616E_15  2.89935E_13
5.82867E_16  _7.77156E_16  _8.99281E_15  _3.61933E_14
2.05391E_15  _7.99361E_15  _5.52336E_15  _7.9714E_14
      0      0      0      0
```

The error is in any case smaller than 10^{-12} , and quite generally smaller than 10^{-13} . Surprisingly good!

As anticipated at the start of this section, this method works only for well-posed matrices. What if we use matrix with all the characteristics of a well-posed matrix but the symmetry? See the following example:

```
A
1  2  3
4  5  6
7  8  _9
V<-s[3]A
S<-sA
V
0.530336  0.349154  0.772551
```

```

0.609599    0.476238    _0.633709
_0.58918    0.807025    0.039722
S

13.9876      0          0
      0    9.44355      0
      0          0    0.408805
)DIV 1
WAS 1
P<-V+.x(S*_0.5)
P

0.141801    0.113619    1.20828
0.162995    0.154973    _0.991132
_0.157535    0.262615    0.0621259
P[;1]<-P[;1]%P[3;1]
P[;2]<-P[;2]%P[3;2]
P[;3]<-P[;3]%P[3;3]
P

_0.900126    0.432643    19.449
_1.03466     0.590115    _15.9536
      1          1          1

```

These were the same procedures followed in the previous case, but the result is not the hoped one:

```

a[2]A

0.718345    15.8213    _0.175977
1.75386     _12.7717    _0.300327
      1          1          1

```

The determination of the diagonal matrix J shows that P is not the right one:

```

J<-(|P|P)+.xA+.xP
J

_7.19805    _5.71175    1.33996
_16.4136    4.59046    _1.43366
0.0335576    _0.129284    _0.392404

```

The matrix is not diagonal and this is not, with all evidence, the right one. So now let's symmetrize it:

```

A1<-A+.x\oA
A1

14    32    _4
32    77    14
_4    14    194
V<-s[3]A1
S<-sA1
V

0.00128725    0.394206    0.919021

```

```

_0.116837    0.912787    _0.391368
_0.99315    _0.106871    0.0472326
S
195.652      0          0
    0    89.1807      0
    0      0    0.167121
P<-V+.x(S*_0.5)
P
9.20279E_05    0.0417434    2.24807
_0.00835288    0.0966572    _0.957347
_0.0710023    _0.0113169    0.115538
P[;1]<-P[;1]%P[3;1]
P[;2]<-P[;2]%P[3;2]
P[;3]<-P[;3]%P[3;3]
P
_0.00129613    _3.6886    19.4573
    0.117642    _8.54098    _8.28597
        1          1          1

```

This matrix is the right one, this time:

```

a[2]A1
_0.00129613    _3.6886    19.4573
    0.117642    _8.54098    _8.28597
        1          1          1

```

Again, let's rebuild the eigenvalues diagonal matrix:

```

J<-(| % | P)+.xA1+.xP
J
195.652      0    _5.21516E_08
    0    89.1807    2.7885E_10
    0      0    0.167121

```

Apart from some noise in the third column, this is the right matrix:

```

aA1
195.652      0          0
    0    89.1807      0
    0      0    0.167121

```

What to say? Well-posed matrices are all easy to be solved, the QR algorithm, the Lanczos algorithm, the Rayleigh iteration and all the method based on decomposition work splendidly with well-posed matrices...

In any case, this shows once more how powerfulness is the SV Decomposition.

13. POLAR DECOMPOSITION

The Polar Decomposition factorizes any matrix A into two matrices called U and P , where U is an orthogonal matrix, and P is a positive semi-definite symmetric matrix, so that:

$$A = U \cdot P$$

This decomposition has the following property: if matrix A is a linear transformation in the space $\mathfrak{R}^n$, the Polar Decomposition separates it into a reflection (or rotation), represented by U , and a scaling of the space, represented by P . In case matrix A is not invertible, U may be left- or right-orthogonal, or may not be orthogonal at all.

The Polar Decomposition derives from the SVD algorithm, and if the SV Decomposition of A is expressed by the equation:

$$A = W \cdot \Sigma \cdot V^T$$

(here we use W because the symbol U is common between the two systems...), then the Polar Decomposition matrices U and P are defined as:

$$U = W \cdot V^T$$

$$P = V \cdot \Sigma \cdot V^T$$

The function `wA` (or `w[1]A`) returns the matrix U , while `w[2]A` returns the matrix P ; the function `w[3]` returns a scalar containing the iterations of the QR algorithm used in the SV Decomposition (thus corresponding to the command `s[7]A`).

The polar decomposition of any matrix A always exists. If A is invertible, the decomposition is unique, and P is positive definite.

The polar decomposition of an invertible matrix can also be expressed as

$$A = P' \cdot U$$

where

$$P' = U \cdot P \cdot U^{-1}$$

The matrix P' is a symmetric positive definite matrix with the same eigenvalues as P but possibly different eigenvectors.

Some examples:

The first involves an invertible matrix:

```

D1
1  2  3
4  5  6
7  8  _9
wD1
_0.573035  0.769343  0.282387
0.683018  0.257914  0.68335
0.452899  0.584459  _0.673268
w[2]D1

```

```

5.32933    5.89221   _1.69709
5.89221    7.50393   _1.40462
_1.69709   _1.40462   11.0067

```

As you can see, the matrix P is symmetric and positive definite. The orthogonality of U is soon calculated, with) FUZZ set to zero, to enhance the error:

```

U+.x| % |U
      1  1.11022E_16      0
      0      1      0
5.55112E_17      0      1
(| % |U)+.xU
      1      0  _8.32667E_17
      0      1  5.55112E_17
_8.32667E_17  _8.32667E_17      1

```

The matrix P' can be expressed in PP as such:

```

PP<-U+.xP+.x| % |U
PP
      1.81281      3.2489  _0.397988
      3.2489      8.12174  0.694281
_0.397988  0.694281      13.9054

```

The rebuild is:

```

PP+.xU
      1   2   3
      4   5   6
      7   8  _9

```

The next example involves a non invertible matrix:

```

D2<-3 3pi9
D2
      1   2   3
      4   5   6
      7   8   9
      P<-w[2]D2
      U<-wD2
      U
_0.586053  0.0558146  0.697682
0.0558146  0.279073  0.502331
0.697682  0.502331  0.30698
      P
      4.52098  4.68842  4.85587
      4.68842  5.52564  6.36286

```

4.85587 6.36286 7.86985

In this case the matrix U is not orthogonal:

```
( | % | U) + .xU

10      _2      _12
 4      _3      _8
 0  _1.25      _2
U+.x | % | U

10      0      0
_1      _3      _1
_14     0      _4
```

Nonetheless, the rebuild is good:

```
U+.xP

1  2  3
4  5  6
7  8  9
```

Let's repeat the building of P' for this non-invertible case:

```
PP<-U+.xP+.x | % | U
PP

7.14426  _10.7164  _0.893033
_59.8332  _53.582   _4.46516
_126.811  _96.4476  _8.0373
```

The matrix P' (here represented by PP) is neither symmetric nor positive definite and, along with the non-orthogonality of U , is a mark of the non-invertibility of D2. The rebuild:

```
PP+.xU

_5.4081  _3.0405  _0.672897
28.9595  _20.5358  _70.0312
63.3271  _38.0312  _139.389
```

is wrong, because this decomposition is only for invertible matrices.

This Polar Decomposition anyway algorithm works even with non-square matrices:

```
M

83  88  42
31  53   9
13  73  19
78  43  39
37  91  26
49  79  23
51   2  80
U<-wM
```

```

P<-w[2]M
U

0.537593      0.309399      0.0250374
0.156398      0.278484     _0.0999703
_0.337963      0.541537      0.248719
0.716825     _0.0364039     _0.00190433
_0.0968764      0.567457      0.161947
0.192415      0.404653     _0.00522353
0.109857     _0.215359      0.949347
P

112.434  68.3542  56.2166
68.3542  163.129  31.2027
56.2166  31.2027  84.8415
U+.xP

83  88  42
31  53   9
13  73  19
78  43  39
37  91  26
49  79  23
51   2  80

```

In this case, the matrix U is left-orthogonal (it has more rows than columns):

```

U+.x| % | U
0.385361      0.167738     _0.00790809      0.374049      0.127545      0.228509      0.0161953
0.167738      0.112008      0.0730885      0.102162      0.126687      0.143305      _0.137699
_0.00790809    0.0730885      0.469342      _0.262448      0.380319      0.152806      0.0823682
0.374049      0.102162      _0.262448      0.515166     _0.0904094      0.123206      0.0847799
0.127545      0.126687      0.380319     _0.0904094      0.357619      0.210137      0.0208948
0.228509      0.143305      0.152806      0.123206      0.210137      0.200795      _0.0709667
0.0161953     _0.137699      0.0823682      0.0847799      0.0208948     _0.0709667      0.959708
(| % | U) + .xU

1  _2.77556E_17      1.9082E_17
2.77556E_17      1  4.33681E_18
1.38778E_17      9.54098E_18      1

```


14. HESSENBERG DECOMPOSITION

A square matrix H is said to be in *Hessenberg reduction* of a matrix A if it has zero values under the sub-diagonal (or over the super-diagonal) and it retains the same eigenvalues of the original matrix A . The solutions of the eigenspace of a Hessenberg matrix can be faster (using appropriate algorithms) because of the zero items. More: the memory amount can be reduced, if the zero items are omitted (of course, this has sense for very wide matrices).

For example, the structure of a 5×5 Hessenberg matrix is

$$\begin{array}{ccccc} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ 0 & \times & \times & \times & \times \\ 0 & 0 & \times & \times & \times \\ 0 & 0 & 0 & \times & \times \end{array}$$

The process of reduction is accomplished through consecutive transformations (called *Householder reflections*) that decompose the original matrix A into two matrices H , the Hessenberg reduction matrix, and Q , which is orthogonal and that enables rebuilding the original matrix as such:

$$A = Q \cdot H \cdot Q^T$$

The algorithm starts by setting Q as the identity matrix, and H as the original matrix A . The algorithm proceeds by analyzing progressively the columns of A under the diagonal, proceeding from left, and make them retain the same Euclidean norm as the column H_k , but built with the first element non null and the rest null:

$$u = \begin{bmatrix} -(H_k H_k)^{0.5} \\ 0 \\ 0 \\ \dots \\ 0 \end{bmatrix}$$

The reflection is operated by subtracting the new column u from H_k and normalizing the result in the column v :

$$v = \frac{H_k - u}{\sqrt{(H_k - u)^T (H_k - u)}}$$

The reflection is then applied on the work-matrix W which is made as such:

$$W = \begin{bmatrix} I & 0 \\ 0 & I - 2 \cdot v \cdot v^T \end{bmatrix}$$

At each step, the Hessenberg matrix H and the orthogonal matrix Q are updated with:

$$\begin{aligned} H &= W \cdot H \cdot W^T \\ Q &= Q \cdot W^T \end{aligned}$$

The process proceeds by matrix assembling and matrix multiplication and is rather fast, because it uses finite processes (usually $n - 2$ iterations, where n is the order of the argument matrix). For instance, on my modest PC, the elaboration of a 30×30 matrix takes 20 ms only.

The operator `h` performs this type of decomposition of matrix A , returning the Hessenberg matrix H with `hA` (or `h [ 1 ] A`) and the orthogonal matrix Q with `h [ 2 ] A`.

For instance, here is an example we've already used in the eigenvalue space determination:

```

M
25      2      _3      4      5
 6      7      _8      9     10
11     12     _13     14     15
16     17     _18     19     20
21     22     _23     24     _25

hM
      25      _5.06446      2.01304      _0.546753      _4.89898
_29.2233      13.5152      13.2999      _2.99488      53.7564
      0      26.9644      _24.8926      _1.73188      1.70836
      0      0      _1.17308      _0.622581      _1.8239
      0      0      0      2.72005E_15      1.44329E_15

h[2]M
      1      0      0      0      0
0      _0.205316      _0.28796      0.841581      _0.408248
0      _0.376412      _0.405251      0.165586      0.816497
0      _0.547509      _0.522542      _0.510409      _0.408248
0      _0.718605      0.692676      0.0616958      _1.249E_16

```

To prove the conservation of the eigenvalues, we execute the eigenvalue operator a on the matrices M, hM:

```

aM
33.6461      0      0      0      0
      0      13.6766      0      0      0
      0      0      0      0      0
      0      0      0      _0.777386      0
      0      0      0      0      _33.5453

ahM
33.6461      0      0      0      0
      0      13.6766      0      0      0
      0      0      0      0      0
      0      0      0      _0.777386      0
      0      0      0      0      _33.5453

```

The rebuild is possible through:

```

(h[2]M) + .x (hM) + .x\oh[2]M
25      2      _3      4      5
 6      7      _8      9     10
11     12     _13     14     15
16     17     _18     19     20
21     22     _23     24     _25

```

Another more complex example is for matrix B:

```

B
7      3      4      _11      _9      _2

```

```

_6      4      _5      7      1      12
_1      _9      2      2      9      1
_8      0      _1      5      0      8
_4      3      _5      7      2      10
6       1      4      _11     _7      _1
hB

```

```

7       7.27607      5.81205      _0.139701      9.0152      7.93634
12.3693      4.13072      18.9685      _1.20707      10.6833      2.41595
0       _7.16034      2.44776      _0.565594      _4.1814      _3.25096
0       0       _8.59877      2.9151      _3.41686      5.72297
0       0       0       1.04644      _2.8351      _10.9792
0       0       0       0       1.41429      5.34152

```

h[2]B

```

1       0       0       0       0       0
0      _0.485071      _0.200797      0.470675      0.292226      0.646108
0      _0.0808452      _0.114383      _0.190923      0.900598      _0.364489
0      _0.646762      _0.474726      _0.509024      _0.280981      _0.1352
0      _0.323381      0.00538706      0.664547      _0.152971      _0.656028
0      0.485071      _0.849237      0.203188      _0.0342968      _0.0322599

```

(h[2]B) + .x (hB) + .x\oh[2]B

```

7       3       4       _11      _9       _2
_6       4       _5       7       1       12
_1       _9       2       2       9       1
_8      _8.88178E_16      _1       5       0       8
_4       3       _5       7       2       10
6       1       4       _11      _7       _1

```

The rebuild is precise up to 10^{-15} , which is negligible, to me (this result is printed with) FUZZ 0).

15. Case Study

Now I will discuss a case study to see the influence of the Hessenberg reduction in the determination of the eigenvalues of a matrix.

The matrix to be studied is this:

$$A = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 2 & 1 & 0 \\ 0 & 0 & 3 & 1 \\ \varepsilon & 0 & 0 & 4 \end{bmatrix}$$

ε is taken to be a very small value $0 \leq \varepsilon \ll 1$; here we will use the values 10^{-30} , 10^{-25} , 10^{-20} , 10^{-15} , 0.0001 and 0.01. We will determine, priorly, the eigenvalues with $\varepsilon = 0$, and with $\varepsilon = 1$, just to see the limit of the action; at first we use D_0 :

```

D
1  0  0  0
0  2  0  0
0  0  3  0
0  0  0  4
hD

```

```

1  0  0  0
0  2  0  0
0  0  3  0
0  0  0  4

```

ahD

```

4  0  0  0
0  3  0  0
0  0  2  0
0  0  0  1

```

aD

```

4  0  0  0
0  3  0  0
0  0  2  0
0  0  0  1

```

The situation for D_1 (with $\varepsilon = 1$) is:

```
D[4;1]<-1
```

D

```

1  0  0  0
0  2  0  0
0  0  3  0
1  0  0  4

```

hD

```

1  0  0  0
-1 4  0  0
0  0  2  0
0  0  0  3

```

ahD

```

4  0  0  0
0  3  0  0
0  0  2  0
0  0  0  1

```

aD

WARNING: CANNOT CALCULATE THIS EIGENSPACE

It's singular that the second D (with $\varepsilon = 1.0$ cannot be calculated, while hD can. This suggests that **the Hessenberg reduction stabilizes the matrix**, the eigenvalues are maintained and the matrix turns to be solvable.

Now that we have analysed D_0 and D_1 , we can see what happens in series:

```
D[4;1]<-0.01
```

D

```

1      0      0      0
0      2      0      0
0      0      3      0
0.01   0      0      4

```

hD

```

      1      0      0      0
    _0.01      4  _1.33227E_15  2.95823E_31
      0  _1.33227E_15      2  _1.11022E_15
      0      0  _1.11022E_15      3
    ahD

  4  0  0  0
  0  3  0  0
  0  0  2  0
  0  0  0  1
    aD
WARNING: CANNOT CALCULATE THIS EIGENSPACE
      D[4;1]<-0.0001
      D
      1      0      0      0
      0      2      0      0
      0      0      3      0
0.0001      0      0      4
    hD

      1      0      0      0
    _0.0001      4  _1.33227E_15  2.95823E_31
      0  _1.33227E_15      2  _1.11022E_15
      0      0  _1.11022E_15      3
    ahD

  4  0  0  0
  0  3  0  0
  0  0  2  0
  0  0  0  1
    aD
WARNING: CANNOT CALCULATE THIS EIGENSPACE
      D[4;1]<-1E_15
      D
      1      0      0      0
      0      2      0      0
      0      0      3      0
1E_15      0      0      4
    hD

      1      0      0      0
    _1E_15      4  _1.33227E_15  2.95823E_31
      0  _1.33227E_15      2  _1.11022E_15
      0      0  _1.11022E_15      3
    ahD

  4  0  0  0
  0  3  0  0
  0  0  2  0
  0  0  0  1
    aD
WARNING: CANNOT CALCULATE THIS EIGENSPACE

```

```

D[4;1]<-1E_20
D
1      0      0      0
0      2      0      0
0      0      3      0
1E_20  0      0      4
hD

      1      0      0      0
_1E_20      4 _1.33227E_15 2.95823E_31
0 _1.33227E_15      2 _1.11022E_15
0      0 _1.11022E_15      3
ahD

4 0 0 0
0 3 0 0
0 0 2 0
0 0 0 1
aD
WARNING: CANNOT CALCULATE THIS EIGENSPACE
D[4;1]<-1E_25
D
1      0      0      0
0      2      0      0
0      0      3      0
1E_25  0      0      4
hD

      1      0      0      0
_1E_25      4 _1.33227E_15 2.95823E_31
0 _1.33227E_15      2 _1.11022E_15
0      0 _1.11022E_15      3
ahD

4 0 0 0
0 3 0 0
0 0 2 0
0 0 0 1
aD
WARNING: CANNOT CALCULATE THIS EIGENSPACE
D[4;1]<-1E_30
D
1      0      0      0
0      2      0      0
0      0      3      0
1E_30  0      0      4
hD

      1      0      0      0
_1E_30      4 _1.33227E_15 2.95823E_31
0 _1.33227E_15      2 _1.11022E_15

```

```

0 0 _1.11022E_15 3
ahD
4 0 0 0
0 3 0 0
0 0 2 0
0 0 0 1
aD
WARNING: CANNOT CALCULATE THIS EIGENSPACE

```

In all cases, the result is the same, the eigenvalues are retained, and the Hessenberg reduction has the same eigenvalues or the original matrix; the algorithm in the operator a, with its guard-warnings, cannot calculate the eigenvalues; I used the JORD program (which has not guard-warnings) to find the eigenvalues of D , with the following constant results:

```

D[4;1]<-1E_30
D
1 0 0 0
0 2 0 0
0 0 3 0
1E_30 0 0 4
JORD D
1 _1
J
4 0 0 0
0 3 0 0
0 0 2 0
0 0 0 1
D[4;1]<-1E_25
D
1 0 0 0
0 2 0 0
0 0 3 0
1E_25 0 0 4
JORD D
1 _1
J
4 0 0 0
0 3 0 0
0 0 2 0
0 0 0 1
D[4;1]<-1E_20
D
1 0 0 0
0 2 0 0
0 0 3 0
1E_20 0 0 4
JORD D
1 _1

```

```

      J
4  0  0  0
0  3  0  0
0  0  2  0
0  0  0  1
      D[4;1]<-1E_15
      D
      1      0      0      0
      0      2      0      0
      0      0      3      0
1E_15      0      0      4
      JORD D
55  _1
      J
4  0  0  0
0  3  0  0
0  0  2  0
0  0  0  1
      D[4;1]<-0.0001
      D
      1      0      0      0
      0      2      0      0
      0      0      3      0
0.0001      0      0      4
      JORD D
18  _1
      J
4  0  0  0
0  3  0  0
0  0  2  0
0  0  0  1
      D[4;1]<-0.01
      D
      1      0      0      0
      0      2      0      0
      0      0      3      0
0.01      0      0      4
      JORD D
13  _1
      J
4  0  0  0
0  3  0  0
0  0  2  0
0  0  0  1

```


16. BALANCING DECOMPOSITION

Balancing a matrix is a preprocessing step while solving the non-symmetric eigenvalue problem for matrices with a high delta in their values. Balancing a matrix reduces the norm of the matrix itself, hopefully improving the accuracy of the computation, because it preserves the eigenvalues.

For a given vector norm $\|\cdot\|$ (usually the classic Frobenius norm), an $n \times n$ (square) matrix A is said to be *balanced* iff for all $i = 1 \dots n$, the norm of its i -th column and the norm of its i -th row are equal.

As a consequence, A and A' are diagonally similar iff there exists a non-singular diagonal matrix D such that the following decompositions hold:

$$A_b = D^{-1} \cdot A \cdot D$$

$$A = D \cdot A_b \cdot D^{-1}$$

Computing D (and/or A_b) such that A_b is balanced is called balancing A , and A_b is called the balanced matrix version of A .

This process does not alter the matrices with small values, unitary matrices or symmetric matrices, because they are already balanced (in this case, $D = I$).

The algorithm used for **apple** is due to Osborne, improved by Parlett and Reinsch, with the modifications of the proposed revision by R. James and J. Langou²¹.

The balanced version, along with a Hessenberg decomposition, is used in the calculation of the eigenvalues of a matrix: finding the balanced Hessenberg reduction of the original matrix and using this in the calculations (they are similar, because they retain the same eigenvalues), the process of determination of the eigenspace should require less time to be completed for the reduced matrix than for the original matrix, but it should be the same.

The function **bA** (or **b [1] A**) returns either the matrix A itself (whenever already balanced) or its balanced version; **b [2]** returns either the identity matrix (if A is already balanced) or the diagonal matrix D .

The presence of the irregular value **only** in the diagonal, does not change the balancing, because the irregular values are both in the column and in the row. For instance, here is a matrix with one very different value in the diagonal, which make the matrix unstable but that won't be changed by the balancing:

```

)FUZZ 0
WAS 12
M
    5      4      2      1
    0  1E_24    _1    _1
   _1      _1      3      0
    1      1    _1      2
bM
    5      4      2      1
    0  1E_24    _1    _1
   _1      _1      3      0
    1      1    _1      2
b[2]M
    1      0      0      0

```

²¹ See also <https://arxiv.org/pdf/1401.5766>

```

0 1 0 0
0 0 1 0
0 0 0 1
(|%|b[2]M)+.xM+.xb[2]M

5      4      2      1
0 1E_24      _1      _1
_1      _1      3      0
1      1      _1      2

```

If the irregular values are elsewhere, the things change:

```

M

5      4 1E_24      1
0      _5      _1      _1
1E+24      _1      3      0
1      1      _1      2
bM

5 4E+07 1E_09 1E+08
0      _5 _1E+08 _10
1E+09 _1E_08      3      0
1E_08 0.1 _1E+07      2
b[2]M

1E_12      0      0      0
0 1E_05      0      0
0      0 1000      0
0      0      0 0.0001
(b[2]M)+.x(bM)+.x|%|b[2]M

5      4      1E_24      1
0      _5      _1      _1
1E+24      _1      3      0
1      1      _1      2

```

But sometimes the rebuild is difficult:

```

M

5      4 1E_24      1
1E+12      _5      _1      _1
1E+24      _1      3      0
1      1 1E_12      2
bM

5 4E+08 1E_08 10000
10000      _5 _1E+08 _0.0001
1E+08 _1E_08      3      0
0.0001 10000      1      2
b[2]M

1E_13      0      0      0

```

```

0  1E_05      0      0
0      0    1000      0
0      0      0  1E_09

```

Apparently all went well, but if we check:

```

M-(b[2]M)+.x(bM)+.x|b[2]M
_8.88178E_16      0  _3.67342E_40      0
_0.00012207      0      0      0
      0      0      0      0
      0      0      0      0
M-(b[2]M)+.x(bM)+.x|b[2]M

```

What appalls me is that 15 out of 16 values are quite perfect (the -8.88×10^{-16} value is the most disappointing), but what to say about the value [2;1]? The error is huge (-1.22×10^{-4})!

So, this algorithm, which is universally known, sometimes fails.

17. MATRIX TESTS

The linear algebra functions need special care, because sometimes some operations can be processed here and not there, for this and not for that. Some definitions are obvious, but some don't, so I report here a short dictionary to illustrate the various cases for the ones that want to recall the matter:

null matrix

A null matrix (any dimension) is a matrix with all zeroes, i.e. no item is different from zero.

diagonal matrix

A diagonal matrix is a square matrix with non-null values only in the main diagonal (from the upper left to the lower right) and zero elsewhere.

square matrix

A square matrix has the same width for the row and column dimensions.

positive definite matrix

A square matrix is called positive definite if it is symmetric and all its eigenvalues are positive.

lower triangular matrix

A square lower triangular matrix has all null values over the main diagonal.

upper triangular matrix

A square upper triangular matrix has all null values under the main diagonal.

symmetric matrix

A square symmetric matrix has all values obeying the condition $M_{ij} = M_{ji}$.

antisymmetric matrix

A square antisymmetric matrix has all values obeying the condition $M_{ij} = -M_{ji}$.

hypo-symmetric matrix

A square hypo-symmetric matrix has all values obeying the condition $M_{ij} = \frac{1}{M_{ji}}$.

invertible matrix

An invertible matrix has determinant different from zero; other check may include rank equal to the order of the matrix, or the invertible matrix M^{-1} obeys to the condition $M^{-1} \cdot M = M \cdot M^{-1} = I$, where I is the identity matrix (see definition). This definition applies only to proper inversions (and returns false for pseudo-inversions and for all non-square cases, where the determinant is null).

mono-valued matrix

A mono-valued matrix (any dimension) has all elements equal to the same value (even zero, and in this case it is also a null matrix).

identity matrix

A square identity matrix has all ones along the main diagonal (from the upper left to the lower right) and zero elsewhere.

So, to detect all these cases, useful in programming, I conceived the dyadic operator $\mathfrak{t}$ that perform these checks, returning always 1 for *true* and 0 for *false*; the operator $X \mathfrak{t} Y$ acts on matrix Y according to the following true results:

- 1 Case of scalar X:
 - X = 0 matrix Y is null
 - X = 1 matrix Y is diagonal
 - X = 2 matrix Y is square
 - X = 3 matrix Y is positive definite
 - X = 4 matrix Y is lower triangular
 - X = 5 matrix Y is upper triangular
 - X = 6 matrix Y is symmetric
 - X = 7 matrix Y is antisymmetric
 - X = 8 matrix Y is hypo-symmetric (it's assumed implicitly) DIV 1)

X = 9 matrix Y is invertible
 X = 10 matrix Y is mono-valued
 X = 11 Y is the identity matrix

2 Case of vector X:

If X is a vector, the test is conducted for every item, provided the vector elements are all in the range 0 to 11 included.

If X is a matrix or if Y is not a matrix, a rank error is raised.

Notes:

- All tests but the #0 and the #10, are for square matrices, and return false for non-square matrices.
- The test #1 for diagonal property satisfies automatically (of course) the tests #4 and #5 for the triangular property.
- The test #8 for the hypo-symmetry uses implicit DIV 1 settings, to evaluate $0 = \frac{1}{0}$ and avoid error messages in the test.
- The test #9 is here for completeness, and it can be substituted by the determinant calculation.
- The test #10 for the mono-valued property checks if the object matrix has all items equal; if this matrix is diagonal, the test is applied only to the diagonal; otherwise it is applied on the whole matrix

Examples:

	M
	5 4 2 1
	0 _5 _1 _1
	_1 _1 3 0
	1 1 _1 2
	0tM
0	1tM
0	2tM
1	3tM
0	4tM
0	5tM
0	6tM
0	7tM
0	8tM
0	9tM
1	10tM
0	

```

    11tM
0
    I
    1  0  0  0
    0  1  0  0
    0  0  1  0
    0  0  0  1
    0tI
0
    1tI
1
    2tI
1
    3tI
1
    4tI
1
    5tI
1
    6tI
1
    7tI
1
    8tI
1
    9tI
1
    10tI
1
    11tI
1
    L
        1      0      0      0      0
    0.24      1      0      0      0
    0.44  1.70552      1      0      0
    0.64  _6.33129  _84.45      1      0
    0.84  3.11656      3    _6E_17      1
    0tL
0
    1tL
0
    2tL
1
    3tL
0
    4tL
1
    5tL
0
    6tL
0
    7tL

```

```

0
      8tL
0
      9tL
1
      10tL
0
      11tL
0

```

The same example run in vector mode:

```

      (_1+i12)tL
0 0 1 0 1 0 0 0 0 1 0 0

```

Or in case of specific targets:

```

      0 2 11tL
0 1 0

```

But use only admitted values!

```

      2 1 12tL
AXIS ERROR
      2 1 12tL
      ^

```

The null matrix case:

```

      N
0 0 0 0
0 0 0 0
0 0 0 0
0 0 0 0
      (_1+i12)tN
1 1 1 0 1 1 1 1 1 0 1 0

```

As seen in the previous example for the identity matrix case, it's cute (but obvious!) to find that the identity matrix I is diagonal, square, positive definite, both lower and upper triangular, symmetric, antisymmetric (because $0 = -0$), hypo-symmetric (because $0 = \frac{1}{0}$), mono-valued and of course the identity matrix too, failing only for the null-matrix case!

The null matrix N , instead, is null, diagonal, square, lower and upper triangular, symmetric, antisymmetric and hypo-symmetric, mono-valued, failing for the positive definite, invertible and identity matrix cases only.

Table of Contents

1.	Introduction	1
2.	Matrix Determinant	5
3.	Matrix Characteristic Polynomial	6
4.	Echelon and Row Reduced Formats	7
5.	Matrix Condition Number	12
5.1.	Using the condition number	13
6.	QR Decomposition	15
6.1.	Least Square Analysis	18
7.	Eigendecomposition	21
7.1.	Determination of complex eigenvalues	30
7.2.	Utility and versatility of eigenvectors and eigenvalues.	32
7.3.	The Rows Geometric Mean Method	34
8.	Cholesky Decomposition	38
8.1.	Applying the Cholesky Decomposition	40
9.	LDL Decomposition	46
9.1.	Applying the LDL Decomposition	47
10.	LU/PLU Decomposition	49
10.1.	Applying the LU/PLU Decomposition	52
11.	LDU Decomposition	57
11.1.	Applying the LDU Decomposition	59
12.	SV Decomposition	60
12.1.	Applying the SV Decomposition	69
13.	Polar Decomposition	77
14.	Hessenberg Decomposition	81
15.	Case Study	83
16.	Balancing Decomposition	89
17.	Matrix Tests	92